



LLM-KGMQA: large language model-augmented multi-hop question-answering system based on knowledge graph in medical field

FeiLong Wang^{1,2} · Donghui Shi^{1,2} · Jose Aguilar^{3,4,5} · Xinyi Cui¹ · Jinsong Jiang^{1,2} · Longjian Shen¹ · Mengya Li¹

Received: 11 July 2024 / Revised: 18 December 2024 / Accepted: 7 March 2025 /

Published online: 21 April 2025

© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2025

Abstract

In response to the problems of poor performance of large language models in specific domains, limited research on knowledge graphs and question-answering systems incorporating large language models, this paper proposed a multi-hop question-answering system framework based on a knowledge graph in the medical field, which was fully augmented by large language models (LLM-KGMQA). The method primarily addressed the problems of entity linking and multi-hop knowledge path reasoning. To address the entity linking problem, an entity fast-linking algorithm was proposed, which categorized entities based on multiple attributes. Then, it used user mentions to obtain the target attribute set of attributes and further narrowed the entity search scope through attribute intersection operations. Finally, for entities that remained too numerous after the intersection, the method suggested using a pre-trained model for similarity calculation and ranking, and to determine the final entity through construction instructions. Regarding multi-hop knowledge path reasoning, the paper proposed a three-step reasoning framework that included an n -hop subgraph construction algorithm, a knowledge fusion algorithm, and a semantics-based knowledge pruning algorithm. In the entity fast-linking experiments, the maximum computational complexity was reduced by 99.90% through intersection operations. Additionally, an evaluation metric called CRA@ n was used alongside the classic nDCG metric. When using the RoBERTa model for similarity calculations, the CRA@ n score reached a maximum of 96.40, the nDCG scores reached a maximum of 99.80, and the entity fast-linking accuracy was 96.60%. In multi-hop knowledge path reasoning, the paper first validated the need for knowledge fusion by constructing three different forms of instructions. Subsequently, experiments were conducted with several large language models, concluded that the GLM4 model showed the best performance in Chinese semantic reasoning. The accuracy rates for GLM4 after pruning were 99.90%, 83.30%, and 86.60% for 1-hop, 2-hop, and 3-hop, respectively, compared to 95.00%, 6.60%, and 5.00% before pruning. The average response time was reduced by 1.36 s, 6.21 s and 27.07 s after pruning compared to before pruning.

Keywords Knowledge graph · Large language model · Entity linking · Multi-hop knowledge reasoning · Question-answering system

1 Introduction

Large pre-trained language models, also known as large language models (LLMs), include hundreds of billions or even trillions of parameters and have achieved promising results on a variety of natural language processing (NLP) tasks [6, 43, 46] while exhibiting emergent abilities [44]. The emergence and application of LLMs have brought many conveniences to people, demonstrating outstanding performance in human-machine dialog, personal assistants, and production practices, making LLMs increasingly widely used in daily life. However, LLMs face several challenges in practical applications. For instance, their interpretability is weak, limiting their application in critical domains such as healthcare and finance. Moreover, the “Hallucination” issues [1, 23] and “toxicity” issues [41] encountered by LLMs in specific domains also greatly hinder their development in industry fields. Techniques like Chain-of-Thought (CoT) [39, 45] have mitigated hallucination, but factual errors still occur.

Fine-tuning [33] is a technique to address the “Hallucination” issues of LLMs, which involves further training and adjusting some or all of the model parameters based on specific tasks or domains after pre-training. Current fine-tuning techniques for LLMs mainly include Adapter Tuning [14], Prefix Tuning [22], P-Tuning [25], Prompt Tuning [21], and Low-Rank Adaptation (LoRA) [15]. These techniques can be categorized into three types: Adapter, Prefix Tuning, and LoRA. Adapter technology inserts small neural network modules, known as adapters between layers of the pre-trained model. These adapters are only trained for efficient fine-tuning of downstream tasks. Based on this, more efficient fine-tuning techniques like AdapterP [31] and Parallel [13] have been developed. Prefix Tuning technology adds K additional trainable prefix markers to the model’s input or hidden layers, achieving efficient fine-tuning by training only these prefix parameters. Based on this, more efficient fine-tuning techniques like P-Tuning and P-Tuning v2 [24] have been developed. LoRA technology learns small parameter low-rank matrices to approximate updates to the model’s weight matrix W , achieving efficient fine-tuning by optimizing the parameters of these low-rank matrices during fine-tuning training. Based on LoRA, researchers have proposed more efficient fine-tuning techniques like AdaLoRA [53] and QLoRA [8].

Although fine-tuning has achieved specific effects on new datasets and tasks, it often leads to a decline in the model’s performance on the pre-training data, even a loss of the original dialog function, and phenomena referred to as catastrophic forgetting [19]. In addition to the computational resource and time costs introduced during the fine-tuning process, another drawback of fine-tuning LLMs is the inability to update knowledge on time, especially in specific domain question-answering, where issues of insufficient coverage of domain knowledge arise.

A knowledge graph (KG) is a graphical representation method for describing various entities and their relationships in the real world. It explains concepts, entities, and relationships in the objective world in a structured form [48]. Leveraging the structure of the KG allows for a better understanding of the relationships between entities, as well as their meanings and usage in different scenarios. In recent years, with the rapid development of artificial intelligence technology, the construction of KGs in the medical field and NLP technology have gradually become research hotspots. As a semantic network, the KG can effectively organize and represent complex medical knowledge, providing a more intuitive and easily understandable form of knowledge representation. In the healthcare field, KGs not only cover a large amount of information about diseases, symptoms, treatment methods, and drugs, but also include medical research findings and clinical trial data [27, 54]. Combining LLMs with

KGs can efficiently utilize of medical knowledge, thereby improving the performance of medical information retrieval and question-answering systems.

In the face of complex KG queries, especially when the number of entity relationships reaches tens of thousands, achieving accurate and efficient entity linking and multi-hop relationship queries becomes particularly important. Traditional question-answering systems typically perform matching-based entity linking and single-hop inference answering with specific entities as objects. Based on the KG, multi-hop question-answering systems can extract and organize information from multiple relevant sources through multiple reasoning steps, gradually approaching the answer to the question, thereby providing more accurate and detailed answers [4, 18].

However, current research on multi-hop question-answering with KGs involves cumbersome steps as well as complex feature embedding and fusion processes. Some studies fail to fully utilize the structured characteristics of the KG and do not effectively link entities and perform multi-hop path reasoning using the KG. Furthermore, multi-hop question-answering based on KGs currently suffers from poor generalization, computational complexity, poor scalability, and poor tolerance for out-of-vocabulary terms in the question-answering process [30, 34].

To address these issues, this paper proposed a framework of multi-hop question-answering system based on a KG, fully augmented by LLMs (LLM-KGMQA), which fully utilizes the structured data of the KG and the strong semantic understanding and reasoning capabilities of LLMs to achieve multi-hop knowledge retrieval and question-answering in medical KGs.

The main contributions of this paper are as follows:

1. For the entity linking problem, this paper proposed an entity fast-linking algorithm that calculated the intersection of entity attributes and used RoBERTa to compute similarity, achieving fast entity linking.
2. Regarding multi-hop knowledge path reasoning, the paper proposed a framework of three-step reasoning that included an n -hop subgraph construction algorithm, a knowledge fusion algorithm, and a semantics-based knowledge pruning algorithm. First, an n -hop subgraph was constructed based on entities. Then, the entity and relationship information within the subgraph were integrated through knowledge fusion techniques. Finally, redundant or unreliable paths were removed through pruning techniques. By searching for knowledge paths through step-by-step reasoning and combining them with LLMs, it is possible to generate fluent and highly readable responses. This method enhances the interpretability of LLMs in specific domain question-answering and simultaneously reduces the probability of “Hallucinations” occurring in the model’s reactions within specific domains.
3. In response to the issue where LLMs exhibited different reasoning effects when faced with instructions that were semantically the same but differed in form, this paper innovatively proposed an approach to integrate knowledge. Through experimentation, the necessity and effectiveness of knowledge integration were demonstrated.
4. Through multiple comparative experiments and ablation studies, the performance of different LLMs was demonstrated, resulting in a better model for Chinese semantic knowledge retrieval and reasoning, providing a technical foundation for future research on Chinese KG-based multi-hop question-answering.
5. A complete process for multi-hop knowledge question-answering was implemented, leveraging graph databases to enable deep integration and reasoning between LLMs and KGs. In this way, this work provides valuable insights and practical guidance for applying knowledge question-answering systems in real-world production scenarios.

2 Relevant work

KGs, as the core of traditional knowledge engineering, has become the most widely adopted method for symbolic knowledge representation and storage [55]. Its simplicity and ease of use, along with its significant effectiveness in applications such as question answering, search, and recommendation, have promoted its widespread use in industry and various academic fields. In research utilizing KGs for multi-hop question answering, the main challenges include accurately linking entities in user questions to the KG and effectively performing multi-hop knowledge reasoning on the graph [51]. Therefore, this section primarily introduces related work on entity linking and multi-hop question-answering.

2.1 Entity linking

Entity linking (EL) [28, 56] is a task in the field of NLP that aims to associate textual entity mentions with corresponding entities in knowledge bases (such as Wikipedia). Entity mentions can be noun phrases, proper nouns, or any textual fragment that refers to an entity. Entities in knowledge bases typically have rich attribute and relational information, making entity linking a fundamental component of information extraction, KG construction, and many other NLP applications.

Jian et al. [58] addressed the issues of Chinese entity linking, the lack of research on linking emerging and unknown entities, and the neglect of Chinese characters and radical information by traditional BERT models. They proposed a zero-shot entity linking model based on Chinese features. It achieved good performance on the Hansel and CLEEK datasets. For issues of interpretability in entity linking, Peng et al. [29] proposed a demonstrative self-training framework of multi-view thinking navigation, which also introduced a passive entity linking task. Additionally, they used generated question-answering datasets for unsupervised transfer learning. Experiments showed that the framework improved baseline performance on multiple datasets. Singh et al. [38] addressed the challenges of medical entity linking in clinical NLP by proposing the NeighBERT model, which extends BERT by encoding entity relationships in the KG. The model demonstrated good performance on two clinical datasets. Cai et al. [4] addressed the difficulty of models in learning diverse and more challenging negative samples in zero-shot entity linking scenarios. They proposed a generative negative sampling method that enables models to learn negative samples better, thereby improving the performance of zero-shot entity linking.

In the food domain, Darius et al. [7] constructed a new knowledge base and developed a hybrid entity linking model that balances performance and reasoning speed by leveraging the strengths of deep learning, traditional machine learning, and unsupervised methods. They demonstrated the effectiveness of the model through experimentation. Shi et al. [36] addressed the challenges of Chinese short-text entity linking by proposing a method that combines various embedding representations. This method integrates the embedding representations of entities and relationships in KG triples, as well as the embedding representations of descriptive text for entities and relationships.

Although the above steps can preliminarily filter out most entities, it is difficult to resolve the issue of incomplete disambiguation due to insufficient contextual information around entity mentions. In addition, as knowledge bases are updated and new entities emerge, entity linking models need to readapt to these changes, particularly when faced with the task of precise entity linking in the large-scale specific domain KGs, where model performance is often unsatisfactory. With the advent of LLMs, Guo et al. [57] proposed constructing prompt

questions by integrating mention context, transforming the similarity calculation between mentions and candidate entities into a scoring mechanism based on prompt questions, and demonstrated the effectiveness of this method through experimentation. However, there are some problems, such as too complex steps, input instruction length limitations of LLMs, high token consumption, and long response time of long text models. Additionally, these methods have limitations in entity linking for out-of-vocabulary terms.

In summary, the current problems of entity linking are as follows:

1. Knowledge base updates and new entities: As knowledge bases are updated and new entities emerge, entity linking models need to continuously adapt to these changes, particularly when achieving precise entity linking in large-scale medical domain KGs.
2. Limited representational capability: The method of word embeddings has limited capability in representing entity mentions and candidate entities, making it difficult to accurately rank candidate entities.
3. Limitations of LLMs: Methods using prompt engineering have problems, such as input instruction length limitations, high token consumption, and long response time for long texts when dealing with large-scale KG nodes.
4. Out-of-vocabulary problem: Entity linking performance for out-of-vocabulary entities is ineffective.

2.2 KG multi-hop question-answering

KG Multi-hop question-answering is a question-answering technique based on the KG that involves multi-step reasoning and path search in the KG to answer complex questions. Compared to single-hop question-answering, multi-hop question-answering requires traversing multiple nodes and relationships, and integrating information from various sources to obtain answers.

Multi-hop question-answering reasoning based on KGs can be categorized into three types: embedding-based multi-hop question-answering [9], logic-based multi-hop question-answering [12], and path-based multi-hop question-answering [49]. The representative systems include EmbedKGQA [34], RGCN [35], BetaE [32], and PRA [11], and they also have their limitations. EmbedKGQA relies on pre-trained embeddings to answer questions, which constrains its capability to handle complex medical domain queries. Moreover, EmbedKGQA cannot generalize well for query types not seen in the training data. RGCN, on the other hand, has high computational complexity and faces efficiency issues when dealing with large-scale KGs. These issues are particularly pronounced in the medical field due to the vast size and complexity of medical KGs. PRA has poor scalability, as the size of the KG increases, PRA has problems with scalability. As the size of the KG increases, PRA becomes difficult to extend because it requires traversing more paths to find answers.

The complementary enhancement of LLMs and the KG can provide artificial intelligence systems with more reliable, comprehensive, and controllable knowledge processing technology. Exploring multi-hop reasoning and question-answering with the combination of LLMs and the KG, researchers mainly discuss three aspects: enhancing LLMs reasoning with the KG, completing the KG with LLMs, and the collaboration of LLMs and the KG.

In terms of enhancing LLMs reasoning with the KG, Gao et al. [10] proposed a new graph model “Dr.Knows” combined with medical KGs. By retrieving specific knowledge paths about disease pathology from the top N cases and inputting them into the LLMs, the model improves the accuracy of the LLMs in diagnostic prediction. Jiang et al. and Wen et al. [47] improved the understanding and processing ability of LLMs for user queries through the

KG, solving the problem of query incompleteness. Andrus et al. [2] introduced a method that enhances the reasoning and predictive abilities of LLMs in specific domains with external dynamic KGs.

In the aspect of completing the KG with LLMs, Lv et al. [26] proposed the PCGK model, which uses manually defined prompt templates and soft prompts for triadic sentence expression, Jiang et al. [16, 17]’s TAGREAL framework automatically generates prompts and utilizes the implicit knowledge in pre-trained language models for completion, Xu et al. [50]’s MPIKGC framework addresses the issues of incomplete entity information and sparse graph linkage by designing various prompt strategies and keyword summarization.

In exploring the effective integration and reasoning of KG representations and linguistic contexts, Zhang et al. [59] designed text classification for information filtering and used Retrieval Augmented Generate (RAG) technology for knowledge base question-answering, followed by fine-tuning for information extraction. Yasunaga et al. [52] proposed a self-supervised pre-training method based on text and KGs, which performed well on various NLP tasks and complex reasoning. Shu et al. [37] significantly improved the prediction accuracy of KG-related tasks by introducing CoT and instruction fine-tuning techniques, showing excellent performance in multi-hop link prediction tasks. Wang et al. [42] proposed aligning factual knowledge and linguistic representations to the same semantic space, demonstrating its effectiveness in NLP applications. Choudhary and Reddy [5] combined KG-based logical reasoning with LLMs, effectively utilizing the reasoning capabilities of LLMs for KG retrieval through logical decomposition prompts.

These studies demonstrate numerous innovations in integrating KGs and LLMs. However, they also share some common drawbacks. For instance, they rely on manually defined prompt templates, which lack automation and generalization capabilities. While they may perform excellently in specific domains, their generality and scalability across different domains are limited. In conclusion, the main problems and shortcomings of KG-based multi-hop question-answering systems are as follows:

1. Insufficient understanding of complex questions, especially for vague or ambiguous questions. The system’s comprehension of natural language is still inadequate.
2. The problem of error accumulation during the reasoning process. In the multi-hop inference process, errors accumulate at each step, leading to significant deviation in the final answer.
3. The problem of System efficiency. The response speed and computational resource consumption of the system need to be optimized.
4. The problem of poor portability. The system lacks universality and needs to be remodeled when facing different domains.
5. Lack of practicality and application scenarios. Many methods only stay at the theoretical level and are difficult to practical fields, lacking practicality and practical.

3 Method

This section first stated the main problems of the study, and then provided specific solutions to the problems.

3.1 Problem statement

This article mainly explored two main tasks in the multi-hop question-answering system: entity linking and multi-hop knowledge path reasoning.

For the entity linking task, this paper proposed an entity fast-linking algorithm that focused on quickly extracting entities from user queries that matched those in the KG, especially in complex scenarios. This study distinguished between two specific cases of entity linking, namely case a (Direct Matching) and case b (Indirect matching), as illustrated in Table 1. The following section presented an introduction to cases a and b:

- Case a (Direct matching). This occurred when the entity terms in the user’s query directly corresponded to entities within the KG. For example, if a user query mentioned “urticaria” which corresponded to the Chinese term “荨麻疹”, the system directly identified “urticaria” as the correct entity.
- Case b (Indirect Matching). This scenario arose when the terms used in the user’s query were out-of-vocabulary or incorrectly spelled, requiring the system to infer the correct entity. For instance, if user query used the term “寻麻疹” instead of the proper term “荨麻疹” for “urticaria”, the system had to recognized and correct the mistake to identify the proper entity.

Multi-hop relational path reasoning primarily focused on constructing an n -hop subgraph connected to the current entity based on entity linking and utilizing LLMs for semantic understanding and reasoning according to user’s intent, while removing other irrelevant edges and nodes. The aim was to achieve rapid and efficient question-answering of medical knowledge while minimizing token consumption. Research on multi-hop knowledge path reasoning encompassed a three-step reasoning framework, which includes the following elements:

1. Construction of the n -hop subgraph. This involved constructing a subgraph of hops for the target entity, transforming the question into a path search problem on the graph.
2. Knowledge fusion. When knowledge paths of the same relation were scattered across different parts of the instructions and exhibit crossovers or logical confusion, LLMs could generate errors in producing answers or retrieving knowledge. The ability of the language models to retrieve and aggregate knowledge could be enhanced by merging knowledge triples of the same relation and organizing them into a single knowledge path. For example, knowledge directly queried from the KG might be: [(‘Urticaria’, ‘Recommended Diet’, ‘Boiled Cabbage’)], [(‘Urticaria’, ‘Recommended Diet’, ‘Boiled Green Bean Puree’)], [(‘Urticaria’, ‘Recommended Diet’, ‘Cold Mixed Water Spinach’)]. After merging with the corresponding algorithm, it became Urticaria → Recommended Diet → Boiled Cabbage, Boiled Green Bean Puree, Cold Mixed Water Spinach.

Table 1 Classification of entity linking research

Case	Example	Correct entity
Case a (Direct Matching)	What are the symptoms of urticaria, and what can be eaten to alleviate it? (荨麻疹)	Urticarial (荨麻疹)
Case b (Indirect Matching)	What are the symptoms of urticaria, and what can be eaten to alleviate it? (寻麻疹)	

3. Pruning of the subgraph. Pruning techniques were used to remove redundant or unreliable paths, thereby improving reasoning efficiency.

3.2 Framework of LLM-KGMQA

The framework diagram of LLM-KGMQA is shown in Fig. 1. The framework first determined whether the user's input question was related to the specific domain. If it was, it proceeded with entity fast-linking to select entities from the KG that match the user's question and return them. In the entity fast-linking stage, the system employed LLMs and pre-trained models for entity linking. Before performing entity fast-linking on the user's input question, the "Attribute Agent" categorized all entity words according to different types of attributes, resulting in a collection of entity attributes. Then, upon receiving a user question, the "Question Agent" classified the question by attributes, performed an intersection operation on the determined attribute sets to obtain a candidate entity set, and checked whether the candidate set was greater than n . If it was less than n , the "Entity Agent" directly selected the final entity. Otherwise, the pre-trained model is used to further compute the similarity between the candidate entities and the user's input, rank the entities based on similarity, select the top L entities, and use the "Entity Agent" to decide the final entity.

After determining the final entity, an n -hop subgraph E was constructed using the entity, and multi-hop path reasoning was performed based on the user's question. The circled parts in the right of Fig. 1 represents the current determined hops, and through such iterative reasoning, the number of nodes within the circles increased step by step, ultimately forming the user's intent graph. Finally, the intention graph derived from the reasoning was organized into instructions and fed into LLMs to generate professional, logical, and vertically-oriented answers.

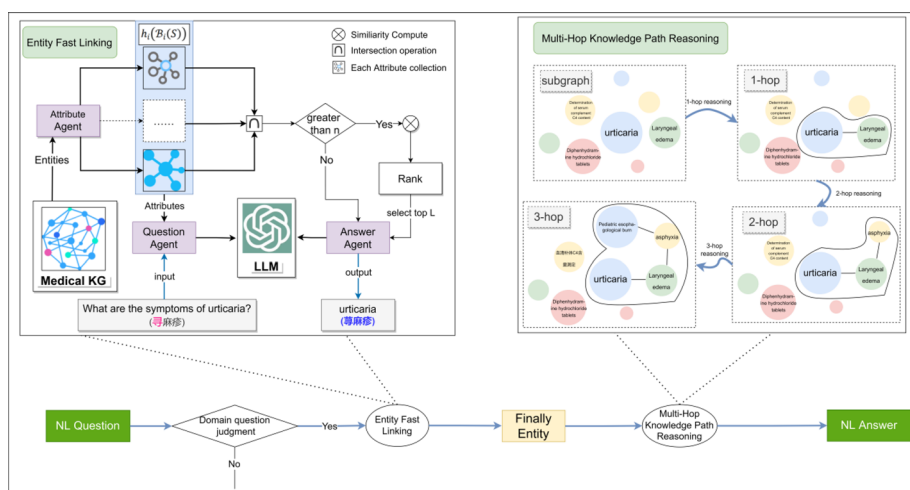


Fig. 1 The framework of LLM-KGMQA

3.3 Entity fast-linking

The process of the entity fast-linking algorithm proposed in this paper was described in Algorithm 1 of Appendix E, which included the handling of two cases: Case a and Case b.

Case a. For the user's input S , the algorithm directly used the Aho-Corasick [20] multi-pattern matching algorithm to match S with the set of entities in the KG. Aho-Corasick consists of a prefix tree, failure pointers, and output pointers. For each character t_i in the text $T = \{t_1 t_2 \cdots t_n\}$, it checked whether there is an edge marked with the character t_i starting from the current node. If such an edge exists, the algorithm moved along that edge to the next node; if it does not exist, the algorithm moved along the path indicated by the failure pointer to the next node. During this process, whenever the algorithm passed through a node, it checked the output pointer of that node. If the output pointer was not empty, it indicated that the node is associated with a certain pattern, and the algorithm added the corresponding pattern to the match result set. This process continued until all characters in the text T had been processed. If the match was successful, the matched entities were added to the final entity list, and the final entity $entity_{final}$ was returned directly (see steps 1–6 in Algorithm 1 of Appendix E).

Case b. If the Aho-Corasick matching result was empty, the algorithm transitioned to the handling process for case b, as described in steps 9–24 of Algorithm 1 of Appendix E. First, all entity sets in the KG were divided into multiple attributes based on different categories. On this basis, the attributes of different categories for S were determined, and their intersections were calculated to narrow down the entity search scope and generate a candidate entity set. Next, it was determined whether the candidate entity set was smaller than the threshold n . If it was smaller than n , instructions were directly constructed, and the final entity was determined by the LLMs. Otherwise, the RoBERTa model was used to calculate the similarity between the entity words in S and the entities in the candidate entity set one by one, selecting the top L entities with the highest similarity. Finally, instructions were constructed and handed over to the LLMs to determine the $entity_{final}$. The following explained the specific methods of this process.

First, each entity in the KG was classified based on different categories. The set of all entities in the KG was denoted as $Entity = \{e_1, e_2, \dots, e_n\}$, and different categories were defined as $\mathcal{C} = \{C_1, C_2, \dots, C_n\}$, where n represents the number of categories. For each category $C_i (i = 1, 2, \dots, n)$, this category contained attributes, denoted as the attribute set $\mathcal{A}_i = \{a_{i1}, a_{i2}, \dots, a_{im_i}\}$. Each attribute $a_{ij} \in \mathcal{A}_i (j = 1, 2, \dots, m_i)$, where m_i represents the number of attributes in category C_i was a condition that an entity needs to satisfy to be classified into this category. The classification function $f_i : Entity \rightarrow \mathcal{A}_i$ was defined to map an entity $e \in Entity$ to the attribute set $\mathcal{A}_i$ in category C_i . That was, $f_i(e) = a_{ij}$. In summary, this can be expressed as Eq. (1).

$$\forall i \in \{1, 2, \dots, n\}, \forall e \in Entity, \exists j \in \{1, 2, \dots, m_i\} \quad (1)$$

This ensures that each entity e to be classified into a specific attribute a_{ij} according to the classification f_i in each category C_i . Next, the entity attributes of the user's input question S were determined, and an intersection operation was performed based on the resulting list of attributes. A function g_i was defined to find the set of attributes in category C_i that meet the conditions based on the S . The output of this function, which satisfies the attribute set of question S , was denoted as $\mathcal{B}_i(S) \subseteq \mathcal{A}_i$. g_i satisfied Eq. (2).

$$g_i(S) = \{a_{ij} \in \mathcal{A}_i | P(a_{ij}, S)\} \quad (2)$$

where $P(a_{ij}, S)$ was a predicate that represents whether the attribute a_{ij} satisfied the question S . The following defined the function h_i to map the attribute set to the entity set, representing which entities have these attributes, the definition of function h_i can be found in Eq. (3).

$$h_i(\mathcal{B}_i(S)) = \{e \in \text{Entity} | f_i(e) \in \mathcal{B}_i(S)\} \quad (3)$$

Specifically, $h_i(\mathcal{B}_i(S))$ was the set of entities that possess any attribute from set $\mathcal{B}_i(S)$ across all classes C_i . Finally, the sets of entities obtained in each category were intersected to obtain the candidate entity set $C\text{Entities} = (e'_1, e'_2, \dots, e'_m)$, as shown in Eq. (4).

$$C\text{Entities} = \bigcap_{i=1}^n h_i(\mathcal{B}_i(S)) \quad (4)$$

Nevertheless, the number of entities might still be excessive following the intersection, exceeding the maximum input length accepted by some LLMs. Furthermore, lengthy text inputs not only result in increased token consumption but also prolong response time. Of greater concern was that when the length of input instruction exceeded a certain threshold, even though the candidate entities did contain the target entity, the model was unable to accurately identify and capture the target entity among the numerous entities due to the input length limitation.

The following determined whether the length of the candidate entity set $C\text{Entities}$ was smaller than n . If it was, the *entity_finall* was judged directly via the instruction $\mathcal{P}_4$ shown in Table 14 in the Appendix A. When the length of the candidate entity set exceeded n , further narrowing was required to improve the model's response speed.

Thus, a similarity calculation function, $\text{Similarity} : S \times C\text{Entities} \rightarrow \mathbb{R}$, was defined to calculate the similarity between the user's question S and each entity e'_i in the candidate entity list $C\text{Entities}$, i.e., $\text{Similarity}(S, e'_i)$. Due to the length and semantic complexity of S , directly using Similarity may result in inaccurate calculations. Therefore, the instruction in $\mathcal{P}_{11}$ was introduced to extract the disease keyword *word* from S , denoted as $K : \text{word} \leftarrow S$. Using *word* instead of S , the similarity calculation function was redefined as $\text{Similarity} : \text{word} \times C\text{Entities} \rightarrow \mathbb{R}$, i.e., $\text{Similarity}(\text{word}, e'_i)$. The RoBERTa pre-trained model was used to obtain their embeddings, v_{word} and $v_{e'_i}$, respectively. The cosine similarity between these embedding vectors was then calculated as the measure of word similarity, as shown in Eq. (5).

$$\text{Similarity}(v_{\text{word}}, v_{e'_i}) = \frac{v_{\text{word}} \cdot v_{e'_i}}{\|v_{\text{word}}\|_2 \|v_{e'_i}\|_2} \quad (5)$$

Here, $v_{\text{word}} \cdot v_{e'_i}$ represented the dot product of the vectors, $\|v_{\text{word}}\|_2$ and $\|v_{e'_i}\|_2$ were the Euclidean norms of the vectors, i.e., the length of the vectors (see steps 12–17 in Algorithm 1 of Appendix E).

After conducting similarity calculations, we sorted the similarity scores and selected the top L entities to construct the instruction $\mathcal{P}_{12}$ shown in Table 14 in the Appendix A for the final entity selection (see steps 17–23 in Algorithm 1 of Appendix E).

3.4 Multi-hop knowledge path reasoning

After entity fast-linking, we first constructed a subgraph for $entity_{final}$, denoted as E , composed of the n -hop knowledge paths from the entity. Then, we used a knowledge fusion algorithm to merge same-type relations among these single-hop knowledge triplets and constructed instructions based on the fused knowledge. This allows us to infer whether the current single-hop knowledge was sufficient to support the user's query needs. If additional knowledge was required, we used the single-hop knowledge paths as a basis to continue the inference, constructing instructions to obtain the next hop knowledge that aligns with the user's intent. During this process, irrelevant knowledge was pruned. This iterative process continued until we obtained the final intent graph E' after pruning the subgraph E .

3.4.1 The algorithm for construction n -hop subgraph E

This paper presented an algorithm for constructing a subgraph E (see Algorithm 2 of Appendix E). The set of nodes was denoted as V , and the set of relationships was denoted as R . The current node was $entity \in V$, the set of visited nodes was $visited \subseteq V$, the current depth was $current_depth \in \mathbb{N}$, and the current path list $path$ contained a list of triplets, each representing a path, $path \subseteq (V \times E \times V)$. The dictionary $pathDict$ stored all paths, with the starting node name as the key and the path list as the value, $pathDict \subseteq (V \times (V \times E \times V)^*)$. The initialization of the variables was the first step. These were the visited nodes set, $visited$, the path list $path$, and the path dictionary $pathDict$. The next step was to ascertain whether the current node had already been visited or whether the maximum depth n had been reached. If either of the aforementioned conditions was met, the current path $path$ should be appended to the path dictionary $pathDict$ and the resulting path dictionary should be returned. In the event that the current node had not been previously visited and the depth had not yet reached the maximum, it was necessary to add the current node to the set of visited nodes, as demonstrated in steps 1–8 of Algorithm 2 of Appendix E. At last, the knowledge query statement should be constructed in order to retrieve all relational edges and tail entity node information, commencing from the current node and concluding with the storage of the aforementioned information in the *result*.

During the process of querying full n -hop path knowledge, some nodes might not have sufficient full path knowledge to reach n -hop. If the node was already the last node in the *path* and *path* was not empty, added the current *path* to the *pathDict* and return (see steps 10–15 in Algorithm 2 of Appendix E). The *result* should be iterated through, with each sub-element encapsulated in a tuple type (including the current node, relationship, and tail node) and added to the list of *path*. The function should be called recursively for the tail node in order to continue the search for the next node. During the recursive call, the depth should be updated and the path list *result* and path dictionary *pathDict* should be passed. Prior to returning, it was necessary to clear the current path list in order to initiate a new path search. Finally, the path dictionary, which stored all hop knowledge paths, must be returned (see steps 16–21 in Algorithm 2 of Appendix E).

For the n -hop knowledge of $entity_{final}$, it was observed that the scale of the constructed subgraph exhibits significant variability. For example, starting from “forehead wrinkles,” the 3-hop full-path knowledge involved 87 entity nodes and 94 unidirectional path relations. In contrast, starting from “urticaria” the 3-hop full-path knowledge included 380 entity nodes and 489 unidirectional path relations, with nearly four times the difference in the number of nodes and edges. This discrepancy highlighted the significant variation in the complexity of

associations between different entities when performing question-answering inference in a KG. To meet these requirements, it was necessary to prune the subgraph E .

During the pruning process, this paper adopted a method of incrementally increasing the effective number of hops in the knowledge paths. Before pruning, it was necessary to integrate the knowledge triplets to enhance the coupling of relevant knowledge and the capabilities of LLMs in semantic understanding and reasoning. To this end, this paper proposed a knowledge fusion algorithm.

3.4.2 The algorithm for knowledge fusion

Next, knowledge triples with the same relationships were merged. $\mathbb{P} : E \rightarrow 2^T$ was defined as a mapping from the set E to the power set 2^T of triples, where T was the set of all possible triples. Each triple $t \in T$ was an ordered pair (h, r, t) , where h was the head entity, r was the relationship, and t was the tail entity. Function $f : E \times 2^T \rightarrow 2^L$ was defined, where L was the set of all possible paths. The function f was defined as follows (see Eq. 6).

$$f(e, \mathbb{P}(e)) = \{l_1, l_2, \dots, l_n\} \quad (6)$$

where l_i was a knowledge path of the form $h \rightarrow r \rightarrow t$, and the process for merging identical relations was detailed in steps 1–12 of Algorithm 3 of Appendix E. All i -hop knowledge triples (where $i < n$) were extracted from the n -hop subgraph E constructed by the *entity*, and the resulting list was denoted as $level_i$. In this list, the merged i -th level knowledge path was referred to as $levelKP_i$, the list used to store t was called T_{list} , the relationship of the previous knowledge was denoted as r_{prev} , and the head entity of the previous knowledge was h_{prev} . $level_i$ was processed by traversal. During this traversal, if the relationship of the current knowledge triple differed from that of the previous knowledge triple and the current knowledge triple was not the first data item in the list, the algorithm added the current knowledge triple to the knowledge path list.

To avoid the t of the previous knowledge being mistakenly included in the next relationship path, the algorithm cleared T_{list} and added the new t to T_{list} (see steps 4–8 of Algorithm 3 of Appendix E). At the same time, to ensure that the last knowledge triple was not missed, when the traversal reached the last data in the list, it was added to the knowledge path list (see steps 9–12 in Algorithm 3 of Appendix E). The above described the processing flow of the algorithm when the relationship of the current knowledge triple was different from the previous one. In the event that the relationship of the current knowledge triple was identical to that of the previous one, the algorithm will proceed to add t to the knowledge path list, updated h_{prev} and r_{prev} to the h and r of the current knowledge triple (see steps 13–18 in Algorithm 3 of Appendix E).

3.4.3 A semantics-based knowledge pruning algorithm

In the multi-hop question-answering research, this paper mainly investigated question-answering scenarios with three hops or fewer. The organized knowledge paths can be used for the LLMs to answer questions in specific domains. However, if the current path information was insufficient to answer the user's question, it was necessary to reason about the next hop knowledge paths in combination with the current knowledge paths and the subgraph E . The instructions for 1-hop, 2-hop, and 3-hop to determine whether the current path information was sufficient to answer the user's question were $\mathcal{P}_5$, $\mathcal{P}_6$, and $\mathcal{P}_7$ shown in Table 14 in the Appendix A.

However, during the inference and judgment process, certain knowledge paths appeared excessively long and had limited substantive significance compared to those solely used to determine the existence of a knowledge path aligned with the user's intent, for example, the path "Urticaria $\rightarrow$ Recommended Medication $\rightarrow$ Loratadine Capsules, Fexofenadine Hydrochloride Tablets, Fengshen Tongsheng Pills, Loratadine Tablets, Cetirizine Hydrochloride Drops, Cetirizine Hydrochloride Tablets, Diphenhydramine Hydrochloride Tablets, Fuyang Granules, Promethazine Hydrochloride Tablets". Therefore, this paper proposed an optimized method for utilizing knowledge paths. Specifically, for the current i -th hop knowledge path $LevelKP_i$, the path was simplified during inference and then the complete path was used when providing the answer. During inference, the first tail entity of the path was retained as the answer, simplifying it to: "Urticaria $\rightarrow$ Recommended Medication $\rightarrow$ Loratadine Capsules". This approach not only achieved the purpose of determining the user's question intent but also allowed the LLMs to focus on a smaller amount of text to capture key information more effectively. Additionally, the reduced input could also reduce the consumption of tokens.

The simplified knowledge path was denoted as $simpleLevelKP_i$, then, we determined whether the current i -th hop knowledge path was sufficient to support the user's answer. If this condition was met, the current knowledge path could be regarded as the final intent graph, and the final answer was provided by constructing the instruction $\mathcal{P}_8$ shown in Table 14 in Appendix A. This process is described in steps 1–6 of Algorithm 4. If it was not sufficient to support the user's answer, we considered using the tail entity t of the most relevant path in the current knowledge path for reasoning to obtain the next hop knowledge path related to the user's question until it was sufficient to support the user's answer (see steps 7–16 in Algorithm 4 of Appendix E). The method for reasoning the next hop knowledge path was as follows:

First, by constructing the instruction $\mathcal{P}_9$ or $\mathcal{P}_{10}$ shown Table 14 in the Appendix A, we reasoned from $LevelKP_I$ to obtain the most relevant path $textPath$ in the user's question S , where $textPath \in L$ (see Eq. 7).

$$textPath^* = f(LevelKP_i, S, n) \quad (7)$$

where f satisfied Eq. 8.

$$f(LevelKP_i, S, n) = \begin{cases} \emptyset, & \text{if not query knowledge path} \\ textPath^*, & \text{else} \end{cases} \quad (8)$$

Once the most pertinent knowledge path to the user's query had been identified, the t from the path was extracted. The next hop knowledge triple $level_j$, which was connected to the t in the subgraph E , was then searched for. If the current recursive depth was less than n , the reasoning process continued with the next hop knowledge path. Otherwise, $level_j$ was merged to obtain the knowledge path $LevelKP_j$, at which point the knowledge path was the final intent graph E .

The algorithm was straightforward in its implementation, utilized LLMs for comprehensive semantic understanding and reasoning, thereby achieved pruning and filtering of the original full n -hop subgraph.

4 Experiments

4.1 Evaluation metrics

This paper builds upon the Hits@ n metric [3] in knowledge engineering, proposing the CRA@ n (Cumulative Ranking Accuracy@ n) evaluation metric. The CRA@ n metric is designed to assess the efficacy of an entity linking algorithm in identifying the most relevant result within the top n recommended results. This metric represents the ranking score of the top n elements in N . The calculation formula for CRA@ n is presented in Eq. (9).

$$CRA@n = \frac{1}{N} \sum_{i=1}^N rank_i \quad (9)$$

where N represents the total number of entities calculated, and $rank_i$ represents the ranking score of the i -th element in the similarity ranking (see Eq. 10).

$$rank_i = \frac{100}{n} \times (n - i + 1) \quad (10)$$

The CRA@ n metric represents an extension and improvement of the existing Hits@ n metric. The latter indicates whether at least one correct answer is included in the top n recommended results. In contrast to Hits@ n , the CRA@ n metric not only checks for the presence of correct answers within the top n results but also calculates the ranking scores of these results and takes their average to measure accuracy. A higher value of this metric indicates that the algorithm is more effective in handling out-of-vocabulary terms and user input errors, providing a more detailed reflection of the model's performance.

To further evaluate the effectiveness of the proposed metric and model in this paper, we also utilize the classical metric nDCG (Normalized Discounted Cumulative Gain) for testing [40]. nDCG, is a widely used evaluation metric for assessing the performance of ranking systems. It measures the relevance and order of recommended results, assigning higher weights to relevant items at higher ranks. By normalizing the scores, nDCG ensures comparability across different datasets, offering a more comprehensive performance assessment.

4.2 Experimental results and analysis

The experimental results of entity fast-linking and multi-hop question-answering reasoning are introduced in Sects. 4.2.1 and 4.2.2, respectively. The experimental model selection and hyperparameter settings for the LLMs are shown in Appendix C.

In the experiment on entity fast-linking, we initially verified the percentage reduction in computational load resulting from the intersection operation in comparison to the original entity grouping. This demonstrated the effectiveness of the intersection operation. The numbers of entities and relationships for each type are provided in Appendix B (see Tables 16 and 17). To determine the optimal model for similarity calculation, we selected the BERT, ALBERT, and RoBERTa models to compute similarity scores. Additionally, to validate the effectiveness of extracting entity keywords from user input before calculating similarity, we compared the similarity scores with and without extracting entity keywords from user input. Finally, we employed a range of LLMs to select the final entities, with the objective of identifying the optimal model.

In the multi-hop knowledge path reasoning experiment, we first adjusted the prompt statements in the instructions in three different scenarios to demonstrate the necessity of the proposed knowledge fusion algorithm (Algorithm 3 of Appendix E). This was done in order to

test the consistency of responses before and after the adjustments. Based on this, we provided the accuracy rates of different LLMs for responses within three hops, offering intuitive results for selecting the best-performing model. Subsequently, an error tracing analysis was conducted on incorrect answers from different models in order to identify the stages where underperforming models made mistakes. Finally, the effectiveness of the pruning algorithm (Algorithm 4 of Appendix E) was validated by comparing the accuracy and average response times before and after pruning.

4.2.1 Entity fast-linking

After classifying the departments and affected areas, the user's input question S was categorized by department and affected area. The classification instructions for department and affected areas were provided in Appendix A Table 14 $\mathcal{P}_2$ and $\mathcal{P}_3$, respectively. Next, the percentage reduction in computational effort was described after classifying diseases by departments and affected areas and taking the intersection of each combination. It was known that there were 54 departments (Table 19 in Appendix B) and 4 affected areas (Table 18 in Appendix B) resulting in 216 possible combinations. However, not all combinations had an intersection; that is, for a combination (A, B), there might be diseases in department A that did not have body part B as their affected area. According to statistical analysis, the number of valid combinations was reduced to 192. The following calculation demonstrated the percentage reduction in the number of computations after intersecting different combinations, compared to the department and affected areas, respectively. The symbols representing different departments and affected areas were listed in Appendix A (see Table 15).

Table 2 illustrates the percentage reduction in computation compared to the original number of departments resulting from the intersection of different combinations. The data in parentheses in Table 2, in the form (A, B), indicates the original set size A and the size of the intersection B after the intersection operation. For example, for (338,2), before taking the intersection, there were 338 entities in department $\mathcal{F}$. After intersecting with $\mathcal{H}$, the intersection size becomes 2. The computational reduction can then be calculated as $(338 - 2)/338$, which equals 99.4%. The first row in the table represents the largest reduction in computational workload, while the last row corresponds to the least reduction in data within the table.

As illustrated in the Table 2, the computation is reduced by 99.40% when a user inquires about the disease "Amenorrhea headache". This indicates that the disease belongs to the gynecology department, which encompasses a total of 338 diseases. After intersecting with the Head set, only two diseases remain, significantly enhancing the speed and efficiency of

Table 2 Comparison with departments (Excerpt)

Combination	Decline percentage	Example
$\mathcal{F} \cap \mathcal{H}$	99.40% (338,2)	Amenorrhea headache
$\mathcal{X} \cap \mathcal{H}$	97.35% (454,12)	Neurogenic vomiting
$\mathcal{n} \cap \mathcal{t}$	86.21% (283,39)	Pediatric simple hematuria
$\mathcal{Y} \cap \mathcal{H}$	77.77% (9,2)	Heart ear syndrome
$\mathcal{S} \cap \mathcal{t}$	67.32% (150,50)	Bladder dyspnea
$\mathcal{K} \cap \mathcal{b}$	50.00% (8,4)	Amoebic liver abscess

entity linking. The greatest reduction in computation time in comparison to other departments was 99.40%, while the lowest was 50.00%.

Table 3 presents the percentage reduction in the number of calculations for different combinations following the intersection operation, in comparison to the original number for affected areas. The data in parentheses in Table 3, in the form (A, B), indicates the original set size A and the size of the intersection B after the intersection operation. For example, for (2060,1), before taking the intersection, there were 2060 entities in areas $\mathcal{H}$. After intersecting with department $\mathcal{G}$, the intersection size becomes 1. The computational reduction can then be calculated as $(2060 - 1)/2060$, which equals 99.95%. The first row in the table represents the largest reduction in computational workload, while the last row corresponds to the least reduction in data within table.

For the disease “Lymphadenopathy”, which affects the Head, the total number of diseases with this affected area is 2060. Following the intersection, only one disease remains, directly yielding the final result, with a calculation reduction of 99.951%. This significantly improves the efficiency of entity linking. The minimum percentage reduction in calculations compared to the affected areas is 60.825% for the disease “Congenital Laryngomalacia”.

In order to assess the efficacy of the methodologies employed, the study calculated scores for two distinct scenarios. The first involved a direct comparison between the user’s input question and the intersection entity list, while the second entailed the extraction of disease entities from the user’s input question and subsequent comparison with the intersection entity list.

Table 4 presents the entity prediction ranking scores obtained by calculating the similarity between user input questions and the pre-trained model. For instance, given a user input query, “how to deal with urticaria”, the similarity between the user query and each entity in the candidate entity set can be directly computed. From the table, it can be observed that the CRA@n series metrics increase as n increases. In the N experimental groups conducted, the similarity score ranking list in the first position for both the BERT and AIBERT models

Table 3 Comparison with affected areas (Excerpt)

Combination	Decline percentage	Example
$\mathcal{H} \cap \mathcal{G}$	99.95% (2060,1)	Lymphadenopathy
$\mathcal{E} \cap \mathcal{N}$	87.24% (2039,260)	constipation
$\mathcal{O} \cap \mathcal{N}$	70.67% (1306,383)	Brenneman syndrome
$\mathcal{H} \cap \mathcal{W}$	60.82% (2060,807)	Congenital Laryngomalacia

Table 4 Sentence-entity ranking scores

Models	Metrics				
	CRA@1	CRA@3	CRA@10	CRA@50	nDCG
AlBert	0.00	0.00	0.00	82.80	10.59
Bert	0.00	6.66	28.00	81.20	37.38
RoBERTa	40.00	66.66	74.00	96.40	66.04

The bold font indicates the maximum value of the respective column’s metric

Table 5 Entity-entity ranking scores

Models	Metrics				
	CRA@1	CRA@3	CRA@10	CRA@50	nDCG
AlBERT	60.00	60.00	72.00	82.80	76.19
BERT	80.00	80.00	80.00	81.20	91.66
RoBERTa	80.00	80.00	82.00	96.40	99.80

The bold font indicates the maximum value of the respective column's metric

is the target entity with a score of 0, while Robert achieves a score of 40.00 in CRA@1, which is better than the other two models. Moreover, both the nDCG metric and CRA@n show consistent performance, with the RoBERTa model achieving the highest nDCG score of 66.04. This highlights the strong performance of the RoBERTa model compared to its peers. In terms of overall performance, the RoBERTa model demonstrated significantly superior results in comparison to the other two models, as evidenced by the obtained scores. The CRA@n series metrics for RoBERTa are 40.00, 66.66, 74.00, and 94.40, respectively, while the ALBERT model achieves a score of 0 in CRA@1, CRA@3, and CRA@10, indicating poor performance in the process of calculating the similarity between user sentences and entity words.

Table 5 presents the results of similarity calculations between the user's question after extracting disease keywords and the intersection entity list. For example, given a user input query "how to deal with urticaria", after extracting the entity keyword "urticaria", we then compute the similarity between "urticaria" and each entity in the candidate entity set. From the table, it is evident that the overall score is significantly higher than that in Table 4. The RoBERTa model continues to demonstrate the highest performance, with CRA@n series metric scores of 80.00, 80.00, 82.00, and 96.40, respectively. Additionally, the nDCG metric achieves the highest score of 99.80 in this experiment, further highlighting the superior performance of the RoBERTa model. The Albert model exhibits the lowest performance among the remaining two models. The BERT model, on the other hand, demonstrates comparable performance to the RoBERTa model in the CRA@1, CRA@3, CRA@10 and nDCG indicators. Nevertheless, the BERT model exhibits slightly inferior performance in the CRA@50 score. In contrast to the direct similarity calculation with the user's sentence, the RoBERTa score after extracting keywords from the user's input question is almost identical in the CRA@50, but is clearly superior in the CRA@1, CRA@3, and CRA@10. In this scenario, the extraction of keywords from the user's input question enables the similarity ranking to identify the target entity with fewer tokens, thus facilitating the accurate identification of the entity by LLMs.

Table 6 illustrates the efficacy of entity linking by different LLMs within the top 50

Table 6 Accuracy of entity linking by different models

Models	Accuracy (%)
Spark3.5	83.30
GPT-3.5	93.30
GLM4	96.60

positions. The top L entities from the ranking or the candidate entity set with less than 50 entities obtained from the intersection operation are employed to construct prompts for the LLMs, which then selects the final entity. The accuracy is subsequently evaluated based on the responses provided by the large language model. It is evident that the GLM4 model achieves the highest accuracy for entity linking of any length within the top 50, while the Spark3.5 model exhibits the lowest accuracy in the entity linking process. This indicates that the Spark3.5 model is inferior to the other two models in semantic understanding and retrieval.

Consequently, in the entity fast-linking process, the GLM4 model was selected for experimental investigation, and the results of the experiment were found to be in accordance with expectations.

4.2.2 Multi-hop question-answering Reasoning

The most critical step in the entire multi-hop reasoning module is the pruning of the subgraph E . This step involves evaluating the coherence of the current knowledge path in relation to the user's query S , i.e., determining whether it can substantiate the user's query and provide an answer, which is a pivotal step. However, the design of the instructions for this step is pivotal to the model's performance. The following section will validate the necessity of Algorithm 3 of Appendix E, which introduces the process of merging knowledge triples into knowledge paths.

The correct instruction is one that includes the correct knowledge path and clearly defines it, with similar relationship paths arranged consecutively and logically coherent. The verification methods include three different transformations of the correct instruction prompt to test their impact on the model. The relationships $A = \{a_1, a_2, \dots, a_n\}$, $B = \{b_1, 2, \dots, b_q\}$, and $C = \{c_1, c_2, \dots, c_m\}$ in the knowledge triples, and the related knowledge paths must be defined. The correct prompt is $\{a_1, a_2, \dots, a_n, b_1, b_2, \dots, b_q, c_1, c_2, \dots, c_m\}$. For the correct prompt, each segment is reordered sequentially (Case a), the knowledge paths are placed in a random order, i.e., $\{a_1, b_2, \dots, c_n, b_1, a_2, \dots, a_n\}$ (Case b), and other unrelated knowledge paths are added to the knowledge path (Case c).

Three distinct scenarios were employed to construct prompt statements, which were subsequently subjected to individual testing to ascertain the accuracy of the GLM4 model's responses under each condition. The results were then recorded and analyzed. Table 7 displays the accuracy for each case.

As observed in Table 7, modifying the segments of the prompt statements has no significant impact on the answer to the question (case a). However, disrupting the knowledge paths within the prompt statements does affect the model's judgment to some extent (case b). Furthermore, directly adding unrelated knowledge to the correct prompt statement significantly reduces the model's effectiveness in the judgment process (case c). This experiment demonstrates

Table 7 Accuracy under different conditions

Case	Accuracy (%)
Case a.	95.00
Case b.	65.00
Case c.	45.00

the necessity and effectiveness of Algorithm 3 of Appendix E, as adjusting and merging knowledge paths effectively improves the model's accuracy in judgment.

In the event of queries pertaining to differing hop counts, it is first necessary to collate and categorize the questions. Subsequently, the aforementioned questions are subjected to testing on a range of models, with each model being tested according to its respective hop count. A question is deemed to have been answered successfully if it can be answered by integrating knowledge paths on a specific model. Conversely, if any issue during testing prevents an answer from being provided at any stage, the question is considered to have been answered unsuccessfully. Consequently, by recording the performance of different question types on different models in this manner, we are able to calculate the accuracy. Table 8 presents the accuracy of responses for each type of question across different models.

Table 8 reveals that the GLM4 model exhibits the highest performance among the five models across three types of questions, with an average accuracy of 89.90%. This is a notable 51.10 percentage point improvement over the GPT-4 model. In contrast, the GPT-4 model exhibits the poorest overall performance, with only acceptable performance observed in 1-hop questions, with accuracies of 16.60% and 13.30% in 2-hop and 3-hop questions, respectively. Although the GPT-4 model demonstrates superior performance in various domains compared to other models, experimental results indicate that its ability to reason and comprehend Chinese semantics is significantly inferior to that of GLM4 (in conjunction with the instructions in Table 14 of the Appendix A and the experimental results in Table 8). For instance, in the 3-hop question "What symptoms are complications of complications of urticaria?" despite successfully retrieving all relevant knowledge paths, the model mostly fails to determine whether they meet the user's query intent. Further experiments suggest that the GPT-4 model fails to delineate the boundaries of 3-hop questions, often misidentifying them as 4-hop questions, such as "What symptoms are complications of complications of complications of urticaria", a problem also observed with the GPT-3.5 model. The Spark3.5 model performs similarly to the GLM4 model for 1-hop questions, but shows increasing accuracy from 2-hop questions onwards. For 2-hop questions, the Spark3.5 model is the second best performing model after the GLM4 model. The Qwen-7B-chat model performed poorly on the remaining two types of problems with an average accuracy of only 34.68%, except for the 1-hop problem where it performed better.

Table 8 shows that the accuracy of the Qwen-7B-chat, Spark3.5, GPT-3.5 and GPT-4 models decreases as the number of hops in the questions increases. The main reason for this phenomenon is that as the number of hops increases, the complexity of the system's processing also increases, leading to a higher probability of errors, which ultimately affects the accuracy of the models. However, unlike other models, the GLM4 model shows higher

Table 8 Accuracy of different models in answering three-hop questions

Models	Accuracy			
	1-hop (%)	2-hop (%)	3-hop (%)	Avg (%)
Qwen-7B-chat	95.65	8.69	0.00	34.68
GPT-4	96.60	16.60	13.30	38.80
GPT3.5	76.60	26.60	15.00	39.40
Spark3.5	93.30	46.70	15.00	51.60
GLM4	99.90	83.30	86.60	89.90

accuracy for 3-hop questions compared to 2-hop questions. This result suggests that in the multi-hop question answering task, the accuracy of the next hop in the proposed framework does not directly depend on the previous hop. Furthermore, the accuracy of the answers does not necessarily decrease with an increasing number of hops. As long as the semantic understanding of the large language model is sufficiently robust, the question-answering scenarios for different numbers of hops can be considered independently. The bar chart showing the accuracy of different models on three types of questions can be seen in Figure 2.

During the testing process in Table 8, we documented whether the model was able to provide an answer by integrating knowledge paths. In cases of failure, we recorded which instruction from Table 14 (see Appendix A) caused the error, and then compiled statistics on the instructions that occurred frequently across models. This analysis was combined with the number of hops in the question to determine the stage of failure. Table 9 displays the stages where the Qwen-7B-chat, GPT-3.5, GPT-4, and Spark3.5 models performed poorly across three types of questions. In the table, “√” indicates that the model made errors in the corresponding stage of the current column, while “-” indicates fewer errors in that stage for the model.

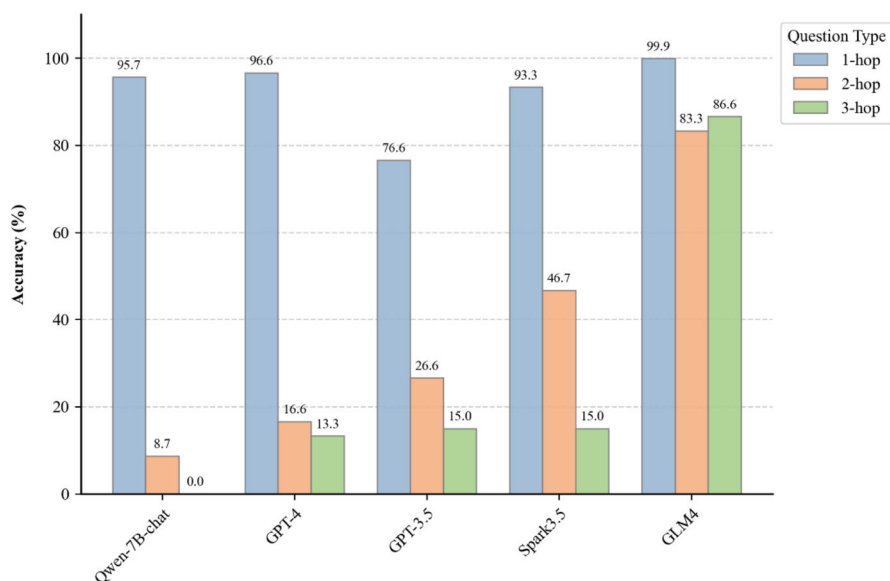


Fig. 2 Bar chart of accuracy rates for different models

Table 9 Stages of error for different models in three-hop questions

Models	Error step		
	1-hop	2-hop	3-hop
GPT-3.5	–	√	√
GPT-4	–	√	√
Qwen-7B-chat	–	√	√
Spark3.5	–	–	√

The Spark3.5 model primarily makes errors in next hop knowledge paths that are consistent with logical reasoning in 2-hop questions, showing a weaker ability to reason about next hop knowledge paths. For example, when answering the question “What foods should be avoided because of the complications of fungal meningitis?”, the model often directly provides dietary restrictions for fungal meningitis rather than relevant information about its complications. This type of reasoning error also occurs in 3-hop questions, and Spark3.5 tends to incorrectly classify 3-hop questions as 1-hop questions.

The GPT-3.5 model performs similarly to Spark3.5 in handling 2-hop questions, suggesting that it has weaker Chinese semantic reasoning capabilities compared to GLM4. GPT-3.5 also has shortcomings in following Chinese instructions. For example, when the instruction clearly requires the model to provide the correct answer directly without additional analysis and responses, GPT-3.5 often shows semantic confusion and repetitive responses in the experiments, which is also a significant reason for its poor performance in handling 2-hop and 3-hop questions.

The Qwen-7B-chat model performs well in handling 1-hop questions, consistently providing accurate judgments and answers for most queries. However, it demonstrates significant limitations in reasoning for 2-hop and 3-hop questions, failing to perform effective multi-hop reasoning. Repeated experiments reveal that the model struggles to accurately determine the number of hops required for a given query. Nearly all 2-hop and 3-hop questions are misclassified as 1-hop questions, leading to incorrect reasoning and responses.

Although the GPT-4 model is widely recognized in the industry for its superior performance, the experiments conducted in this study show that the GPT-4 model accessed via the API has shortcomings in terms of knowledge retrieval and reasoning capabilities in the Chinese language domain compared to the GLM4 model. In particular, GPT-4 tends to be confused when dealing with nested knowledge in Chinese semantics. For example, the question “What are the complications of the complications of disease A?” is often misinterpreted by GPT-4 as “What are the complications of the complications of the complications of disease A?”

To validate the effectiveness of the proposed pruning algorithm, Table 10 shows the comparison of average accuracy and average response time between the pruned and non-pruned treatments using the GLM4 model for 1-hop, 2-hop and 3-hop questions. The method of calculating accuracy here is identical to that used in Table 8, where non-pruning refers to using all triples found within a three-hop range from the KG directly as input instructions and then recording whether the answers were successful. The calculation of average response time involves recording the time taken by models that successfully returned answers, starting from

Table 10 Ablation experiment

Models	<i>n</i> -hop	Whether to prune	Accuracy (%)	Average response time (s)
GLM4	1-hop	✓	99.90	11.71
		×	95.00	13.07
	2-hop	✓	83.30	13.84
		×	6.60	20.05
	3-hop	✓	86.60	13.68
		×	5.00	40.76

the initiation of the query in the KG search to the return of the answer, and then calculating the average time for successful responses.

Table 10 shows that when non-pruning is performed and the instruction becomes too long, the GLM4 model's ability to follow instructions decreases significantly, and it fails to return results according to the user's needs when querying the most relevant knowledge path. In addition, the model's performance in retrieving related knowledge in long texts is significantly inferior to that after pruning, especially in determining whether the current knowledge can find a corresponding path in the knowledge path. There is also some difference in the average response time between pruning and non-pruning. For 1-hop questions the difference is relatively small, but for 2-hop and 3-hop questions the difference becomes more pronounced.

After pruning, the model shows significant improvements in both accuracy and average response time. While there are no significant differences in accuracy and average response time for 1-hop questions, the effects are evident for questions with two or more hops, confirming the effectiveness of the pruning process.

Figure 3 shows the distribution of response times for the GLM4 model on KG-based 1-hop, 2-hop and 3-hop questions. The violin plots, which combine the features of box plots and density plots, not only show the range of quartiles, but also reveal the shape of the data distribution through symmetric kernel density estimation. For 1-hop questions, the median response time is approximately 11.5 s, with a quartile range of 10–13 s. The kernel density estimate shows a distribution that is approximately normal, but extended at both ends. For 2-hop questions, the median response time is about 14 s, with a interquartile range of 13–16 s. The shape of the density distribution shows a sharp peak in the higher response time range, suggesting that most response times are concentrated around the median. For 3-hop questions, the median response time is approximately 13.5 s, with a quartile range of 12–15 s.

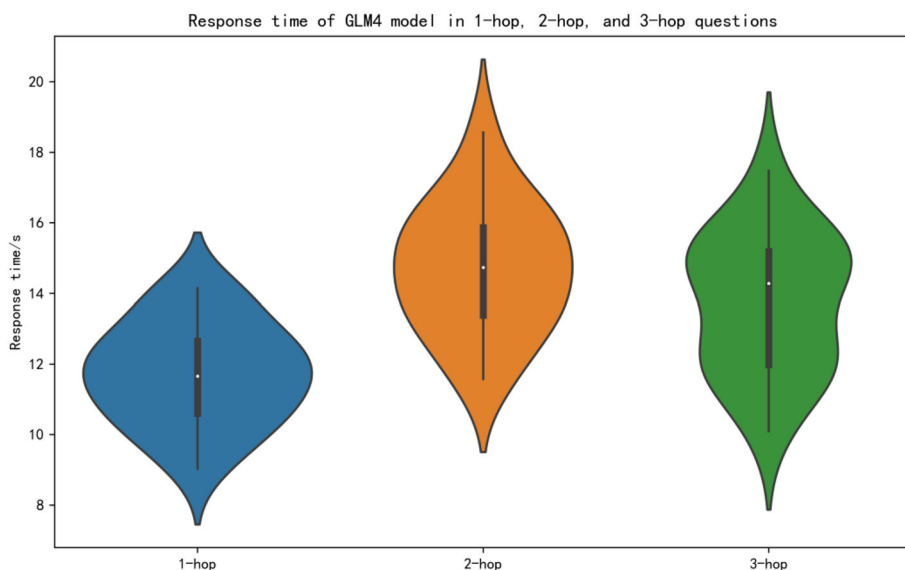


Fig. 3 Violin plots of response time for GLM4 model in three types of questions

The shape of the distribution is similar to that of the 2-hop questions, with a concentrated peak and longer tails.

From Figure 3 it can be seen that the GLM4 model shows certain differences and trends in response time when dealing with different hop questions. Firstly, there is a tendency for the response time to increase with the number of hops. The response time for 1-hop questions is the shortest, indicating that the model can generate answers relatively quickly when directly querying single-hop relationships. The response time for 2-hop and 3-hop questions increases significantly, which is expected as the complexity of the query increases, requiring the model to handle more nodes and relationships, thus increasing the computation time. Secondly, in terms of the stability of the model's response time distribution, the response time distribution for 1-hop questions is more concentrated, suggesting that the model's response time is relatively stable in simple query scenarios. The response time distribution for 2-hop and 3-hop queries is broader, especially with longer tails in the high response time region, indicating that the model's response time can vary significantly when handling more complex queries. Finally, in the high response time tails, there are a few response times that are significantly higher than the average for 2-hop and 3-hop questions. This may be due to individual queries involving a large number of nodes in the KG and a higher complexity of relationships between nodes.

Figure 3 effectively illustrates the performance differences of the GLM4 model for query tasks of different complexity. From the violin plots, we can intuitively see that as the complexity of the query increases, so does the response time, and the distribution of response times for complex query tasks is more spread out. These results provide an important reference for further optimization and improvement of the model's performance in complex query scenarios. The effects of LLM-KGMQA and the other LLMs models for answering the same question are shown in Appendix D (see Table 21 in this Appendix).

4.3 Comparative Experiment

To further demonstrate the effectiveness of the proposed model, we conducted a comparative experiment with HyKGE, a study most relevant to our research [58]. HyKGE utilizes the “gte_sentence_embedding” model for entity linking and the “bge-rerank-large” model for re-ranking in its knowledge graph retrieval module. The similarities and differences between the two approaches are discussed below.

In the entity linking test, HyKGE calculates the similarity between a user-provided query sentence about a disease and all disease entity terms in the knowledge graph, ranks the entities, and selects the most similar entity term as the linking result. The accuracy of entity linking is then evaluated based on this selection. The entity linking accuracy of both the proposed model and HyKGE is shown in Table 11.

Experimental results reveal that the “gte_sentence_embedding” model in HyKGE performs well in cases where the user input closely matches the target entity term (i.e., Case A described in Section 3.1 of this paper) and where the input is error-free. Under such

Table 11 Comparison results of entity linking

Model	Entity linking/%
HyKGE	85.29
KGMQA	96.60

Table 12 Comparison results of knowledge graphs retrieval

Methods	Models	Accuracy/%
HyKGE	bge-rerank-large	30.43
LLM-KGMQA	GPT-3.5-turbo	52.17
	Qwen-7B-chat	65.21
	GLM4-flash	78.26

Table 13 Comparison results of HyKGE and LLM-KGMQA

Model	Entity linking/%	Knowledge graph retrieval/%
HyKGE	85.29	30.43
LLM-KGMQA	96.60	78.26

conditions, HyKGE demonstrates slightly better response speed compared to the proposed model. However, when the user input contains errors (e.g., typos or incorrect terms), the “gte_sentence_embedding” model fails to establish correct entity links. In contrast, the proposed model consistently links to the correct target entity. For instance, when the user mistakenly inputs “片头痛” (incorrect term) instead of “偏头痛”, the GTE model links the input to “房事头疼症” (incorrect entity). However, the proposed model accurately links the input to the intended entity, “偏头痛”.

In the knowledge graph retrieval module, we compared the performance of the proposed model, which utilizes a specific LLMs, with that of HyKGE, which employs the “bge-rerank-large” model. This comparison involved constructing a semantic relationship between the user’s input query and the relevant knowledge paths retrieved from the knowledge graph.

The test results of various LLMs and the “bge-rerank-large” model are summarized in Table 12. As shown in the Table 12, the “bge-rerank-large” model exhibited the poorest performance in the experiments, failing to effectively capture the user’s intent and consistently yielding subpar results across multiple tests. The second-lowest performance was observed with the GPT-3.5-turbo model, which underperformed in a Chinese semantic environment compared to the Qwen-7B-chat model. Notably, the GLM4-flash model achieved the best performance, reaching an accuracy of 78.26%.

Table 13 presents the results of the HyKGE model and the proposed LLM-KGMQA model in both the entity linking stage and the knowledge graph retrieval module. As shown in the table, the proposed model significantly outperforms the HyKGE model in both stages, demonstrating the effectiveness of the proposed approach.

5 Discussion of the results

The results indicated that the proposed system performed well in medical question-answering tasks, demonstrating significant advantages in accuracy and medical domain relevance compared to general LLMs. Key features of the research methodology included:

1. **End-to-End Question-Answering System:** In this study, LLMs were used to construct an end-to-end question-answering system, eliminating the need for separate modeling

- of sub-tasks within multi-hop question-answering systems as required by traditional methods. The inference capabilities of LLMs effectively improved the accuracy and efficiency of both the knowledge retrieval and question answering processes.
2. **Deep Integration of KGs and LLMs:** The study successfully exploited the knowledge and logical structures inherent in KGs and accurately transferred this information to LLMs to achieve a deep integration between the two.
 3. **Token Consumption Optimization:** Throughout the construction of the end-to-end question-answering system, the study carefully considered the issue of token consumption. In the entity association phase, the range of candidate entities was progressively narrowed by intersection calculation and similarity evaluation, thereby reducing unnecessary token consumption. In the multi-hop question-answering reasoning phase, dynamic pruning and knowledge fusion techniques were applied to n -hop subgraphs, minimizing the retrieval of irrelevant knowledge and thus improving retrieval efficiency.
 4. **Simple and User-Friendly Method:** The proposed method was straightforward and easy to understand, avoiding the need for separate modeling of individual sub-tasks, which reduced the likelihood of error propagation between sub-tasks.

6 Conclusion

This paper addressed issues such as poor performance and hallucinations exhibited by LLMs in medical domains. We proposed a medical multi-hop question-answering system that combined LLMs with KGs, LLM-KGMQA. By analyzing the current state of KG-based multi-hop question answering systems, we proposed an entity fast-linking algorithm and a three-step multi-hop knowledge path reasoning framework.

In the entity linking task, an entity fast-linking algorithm was proposed. This algorithm ranked entities through multi-attribute classification, attribute intersection operations, and similarity computations using a pre-trained model to achieve accurate linking. In the experimental evaluation of the entity fast-linking algorithm, the paper proposed the CRA@ n evaluation metric, calculated ranking scores for different pre-trained models, quantifies the percentage reduction in the volume of entity intersection computation, and finally evaluated the accuracy of the entity fast-linking algorithm.

In the multi-hop knowledge path reasoning task, a three-step reasoning framework was proposed, which included subgraph construction, knowledge fusion, and pruning techniques. For the experimental evaluation of the multi-hop knowledge path reasoning task, the paper first validated the necessity of knowledge fusion by constructing three different forms of instructions. Subsequently, it evaluated the accuracy of the LLM-KGMQA model on 1-hop, 2-hop, and 3-hop paths using different LLMs. Lastly, to verify the effectiveness of the pruning algorithm, the study conducted ablation experiments on the best-performing model and compared the impact of different pruning strategies on model performance.

Although LLM-KGMQA has made some progress in medical question answering, it still has some limitations. The entity fast-linking algorithm requires manual refinement of classes and attributes in various scenarios with different entity characteristics and semantic complexity. In addition, the current framework is predominantly focused on Chinese medical texts, requiring adaptations for non-Chinese languages or cross-lingual applications.

Future research directions to enhance the effectiveness and robustness of LLM-KGMQA could focus on the following aspects:

1. Research on adaptive learning techniques to continuously update KGs and refine entity linking algorithms, thereby improving accuracy across diverse medical datasets.
2. Integration of dynamic contextual embeddings and fine-tuning strategies tailored for medical terminologies to enhance the model's capability in handling complex, context-rich queries.
3. Exploration of multimodal approaches that combine textual data with visual or sensor data, opening new possibilities for comprehensive healthcare applications.

The work presented in this paper aims to provide healthcare professionals and patients with an accurate tool for knowledge retrieval and question answering, facilitating the effective application of structured medical knowledge. In addition, this research provides new insights and approaches for studying question-answering systems in vertical domains based on KGs and LLMs, paving the way for further exploration of “neural” and “symbolic” integration.

Appendix A: Table of instructions and mathematical symbols

See Tables 14 and 15.

Table 14 All instructions

Symbol	Instructions
$\mathcal{P}_1$	Please classify the affected areas of this disease based on the entered disease name and your existing knowledge. The affected areas in this task are divided into four categories: the neck and above are referred to as “Head”, the waist to neck area is referred to as “Body”, the waist below is referred to as “Tail”, and unknown categories are referred to as “Other” Please return the answer directly in the answer, and note that your answer can only be selected from Head, Body, Tail, or Other The disease name is {disease}
$\mathcal{P}_2$	Please provide the corresponding department for medical treatment based on the disease-related questions entered by the user. Please note that departments can only be queried from the provided list and can only provide answers directly without the need for additional answers and analysis The user question is: {question} Department List: {KESHI}
$\mathcal{P}_3$	Please classify the affected area of this sentence containing disease information based on the disease-related questions entered by the user and your existing knowledge. The affected areas in this task are divided into four categories: the neck and above are referred to as “Head”, the waist to neck area is referred to as “Body”, the waist below is referred to as “Tail”, and unknown categories are referred to as “Other” Please return the answer directly in the answer, and note that your answer can only be selected from Head, Body, Tail, or Other The user question is: {question}
$\mathcal{P}_4$	Now I need to search for the entity that best matches the user's question based on the sentences they input about the disease and the list of candidate entities. Please note that your answer can only be returned from the list of candidate entities and should be answered directly without the need for additional responses or analysis The user question is: {question} Candidate list: {Inter_list}

Table 14 (continued)

Symbol	Instructions
$\mathcal{P}_5$	Please determine whether all user questions can be directly found from the corresponding knowledge path based on their specific questions. If each question can find a corresponding knowledge path in the knowledge path, please answer “yes”. If at least one question cannot find a corresponding knowledge path in the knowledge path, please answer “No” The user question is: {question} The current knowledge path is: {sample_level1KP}. Please note that the shape $A \rightarrow r \rightarrow B$ indicates that the r of A is B. For example, urticaria $\rightarrow$ complications $\rightarrow$ laryngeal edema indicates that the complication of urticaria is laryngeal edema Additionally, please return the analyzed results directly, either “yes” or “no”, without the need for additional analysis and response.
$\mathcal{P}_6$	Please infer based on the user’s question, combined with the current knowledge of the first and second level paths, to determine whether the user’s question can find the corresponding knowledge path from the following knowledge paths. If the user has multiple questions, each question can find its corresponding knowledge path in the knowledge path. Please answer “Yes”. If at least one of the user’s multiple questions cannot find a corresponding knowledge path in the knowledge path, please answer “No”. Let’s think step by step. The user question is: {question} The current level one path knowledge is: {text1} The current knowledge of the secondary path is: {sample_level2KP} Please note that the shape $A \rightarrow r \rightarrow B$ indicates that the r of A is B. For example, urticaria $\rightarrow$ complications $\rightarrow$ laryngeal edema indicates that the complication of urticaria is laryngeal edema Additionally, please return “yes” or “no” directly without the need for additional analysis and response.
$\mathcal{P}_7$	Please infer step by step based on the user’s specific problem, combined with the current knowledge of the first, second, and third level paths, to determine whether the user’s problem can find the corresponding knowledge path from the knowledge path. If each question can find a corresponding knowledge path in the knowledge path, please answer “yes”. If at least one question cannot find a corresponding knowledge path in the knowledge path, please answer “No” The user question is: {question} The determined first level path knowledge is: {Work} The confirmed secondary path knowledge is: {text2} The current three-level path knowledge is: {sample_level3KP} Let’s think step by step, please note that the shape $A \rightarrow r \rightarrow B$ indicates that the r of A is B. For example, urticaria $\rightarrow$ complications $\rightarrow$ laryngeal edema indicates that the complication of urticaria is laryngeal edema Additionally, please return “yes” or “no” directly without the need for additional analysis and response.
$\mathcal{P}_8$	Please answer the user’s questions based on their input and your own knowledge reserve, with the requirement of providing as detailed answers as possible, while paying attention to the logical order of language and being in line with the context The user question is: {question}, The knowledge triplet retrieved from the knowledge graph is: {knowledge}
$\mathcal{P}_9$	Please infer the next level of knowledge path from the given list in the order of user questions, and select a logical element to directly return. Please note that the returned content must be the elements in the list and cannot be changed arbitrarily, and it meets the requirements of hierarchical reasoning User question: {question} Let’s think step by step, Select the logical next level path information and select from the following list to return: {level1KP} Additionally, please do not return the list format and directly return the elements in the list without the need for additional analysis and response.

Table 14 (continued)

Symbol	Instructions
$\mathcal{P}_{10}$	Please infer the next level knowledge path according to the order of user questions and the first level path information, and select the logical option from the given list to directly return without outputting any other content. Please note that the returned content must be elements from the list and meet the requirements of hierarchical reasoning First level path information: (Work) User question: {question} Choose the logical next level path information, because I will search for the next level knowledge based on the knowledge path you have chosen, select from the following list, and return: {level2KP} Let's think step by step, please note that the shape $A \rightarrow r \rightarrow B$ indicates that the r of A is B. For example, urticaria $\rightarrow$ complications $\rightarrow$ laryngeal edema indicates that the complication of urticaria is laryngeal edema Additionally, please return the string answer directly and do not return data in the format of a list, as there is no need for additional analysis or response.
$\mathcal{P}_{11}$	I now need to match the similarity between the disease name entered by the user and the existing disease name, and find the disease name with the highest similarity to the user input. However, the sentences entered by the user may contain irrelevant information. Please analyze the user's input to extract key disease words, so that I can calculate similarity Attention: Please return the key words you extracted directly and they must be related to the disease name, without any additional response or analysis. The returned format is just a string The sentence entered by the user is: {question}
$\mathcal{P}_{12}$	Now I need to search for the entity that best matches the user's question based on the sentences they input about the disease and the list of candidate entities. Please note that your answer can only be returned from the list of candidate entities and should be answered directly without the need for additional responses or analysis The user question is: {question} Candidate list: {top_20_word}

Table 15 Symbol representation of departments and body parts

Name	Symbol	Name	Symbol
Gynecology	$\mathcal{F}$	Genetics Department	$\mathcal{Y}$
Gastroenterology Department	$\mathcal{X}$	Nephrology Department	$\mathcal{S}$
Pentatology	$\mathcal{W}$	Rehabilitation Department	$\mathcal{K}$
Pediatric Internal Medicine	$\mathcal{n}$	Internal medicine	$\mathcal{N}$
Infectious Diseases Department	$\mathcal{G}$	Head	$\mathcal{h}$
Body	$\mathcal{b}$	Tail	$\mathcal{t}$
Other	$\mathcal{o}$		

Appendix B: Entity and relationship quantity statistics

This paper employed an open medical dataset (<https://github.com/liuhuanyong/QASystemOnMedicalKG>), which underwent data cleaning and processing to construct a medical KG containing 7 types of entities and 10 types of relationships. The graph comprised 44,000 entities and 300,000 entity relationships. The definitions and counts of entity types and relationship types were presented in Tables 16 and 17.

Table 16 Entity type and quantity statistics

Entity type	Number	Example
Check	3353	Bronchography; Arthroscopic examination
Department	54	Plastic surgery and beauty department; burn department
Disease	8807	Thromboangiitis obliterans; Thoracic descending aortic aneurysm
Drug	3828	Jingwanhong Hemorrhoids Cream; Brinzolamide Eye Drops
Food	4870	Beef ball soup with tomato and pickles; Stewed Lamb with Bamboo Shoots
Producer	17,201	Tongyao Pharmaceutical Penicillin V Potassium Tablets; Qingyang Acetate Dexamethasone Tablets
Symptom	5998	Breast tissue hypertrophy; deep hemorrhage of brain parenchyma

Table 17 Relationship type definitions and quantity statistics

Relation type	Number	Example
belongs_to	8844	<Gynecology, belongs_to obstetrics and gynecology>
common_drug	14,649	<Yang Qiang, common_drug, phentolamine mesylate dispersible tablets>
do_eat	22,238	<Thoracic vertebrae fracture, do_eat, black fish>
drugs_of	17,315	<Penicillin V Potassium Tablets, drugs_of, General Pharmaceutical Penicillin V Potassium Tablets>
need_check	39,422	<Unilateral emphysema, need_check, bronchography>
no_eat	22,247	<Lip disease, no_eat, almonds>
recommand_drug	59,467	<Mixed hemorrhoids, recommand_drug, Jingwanhong Hemorrhoid Cream>
recommand_eat	40,221	<hydrocele, recommand_eat, tomato and pickled beef ball soup>
has_symptom	5998	<Early breast cancer, has_symptom, breast tissue hypertrophy>
acompany_with	12,029	<Lower limb communication venous valve insufficiency, accompan_y_with, thromboangiitis obliterans>

According to the proposed method, this paper classified diseases by departments and affected areas. The affected areas were categorized into four types: Head (diseases above the neck), Body (diseases below the neck to above the waist), Tail (diseases below the waist), and Other (diseases that could not be classified by location). The classification instructions for the affected area were provided in Appendix Table 14 $\mathcal{P}_1$. The original JSON dataset was processed in order to obtain classification information for 54 different departments. Following the categorization of disease entities by departments and affected area, a quantitative analysis was performed on each entity. The quantitative analysis for affected areas is presented in Table 18, and the classification quantity statistics for departments are presented in Table 19.

Table 18 Quantitative analysis of affected areas

Area	Number	Area	Number
Head	2060	Tail	2039
Body	3406	Other	1306

Table 19 Quantitative analysis of departments (Partial)

Department	Number	Department	Number
Internal Medicine	2385	Surgery	1703
Orthopedic Surgery	607	Obstetrics and Gynecology	469
Pediatrics	795	General Surgery Department	418
Emergency Department	205	Cardiology Department	328
Oncology Department	533	Neurology Department	386
Medical Oncology	138	Tumor Surgery	394
Pentatology	916	Ophthalmology	482
Rheumatology and Immunology Department	172	Gastroenterology Department	454
Dermatology Department	671	Blood Specialty	330
Nephrology Department	153	Traditional Chinese Medicine Department	486
Urology Department	155	Urology Department	11
Stomatology	167	Psychology Department	36

Appendix C: Experimental model selection and hyperparameter settings

The following LLMs were employed in the experiments: Spark3.5, GPT-3.5, GLM4, and GPT-4. Spark3.5 represents version 3.5 of iFlytek's Spark Cognitive LLMs, which is a cognitive intelligence large model constructed by iFlytek Co.LTD. GPT-3.5 and GPT-4 are constructed using the Transformer architecture, which is based on an encoder-decoder architecture comprising self-attention layers and feedforward neural network layers. GPT is an autoregressive language model that operates in a decoder-only manner. GLM4 is an open-source conversational language model based on the general language model (GLM) architecture, which draws on the design ideas of ChatGPT. It has incorporated code pre-training into the base model GLM-130B and achieved human intent alignment through supervised fine-tuning and other techniques.

The Common hyperparameter settings for all LLMs are shown in Table 20. The “temperature” parameter controls the randomness of the model's output and must be a positive number within the range of 0 to 1. A higher value will make the model's output more random, while a lower value will make the output more stable. The “frequency_penalty” parameter penalizes tokens based on their frequency of occurrence in the text so far, reducing the likelihood of the model repeating the same line verbatim. This parameter is specific to the GPT model and ranges from -2 to 2 . In order to encourage the model to avoid repetitive content while considering the richness of content in knowledge-based question-answering, a value of

Table 20 Hyperparameter settings for large language models

Name	Value
temperature	0.2
frequency_penalty	1.5
max_tokens	1000

1.5 was chosen for this experiment. The “max_tokens” parameter represents the maximum length of the model’s returned results.

The experimental equipment configuration and environment of this project are: 16 vCPU Intel(R) Xeon(R) Platinum 8481C; GPU: RTX 4090D(24GB) * 1; Internalmemory:80GB; Python3.8(ubuntu20.04); Transformer:4.46.3; Torch:2.0.0, Cuda 11.8.

Appendix D: Experimental results and display

Table 21 presents a comparison of responses among three models, including the model employed in this paper. With regard to user queries, all three models concur that laryngeal edema represents a complication of urticaria, particularly when accompanied by angioedema. However, they diverge in their descriptions of symptoms and treatment plans. The model presented in this paper provides a comprehensive account of symptoms, including hoarseness, difficulty breathing, diffuse mucosal swelling in the throat, neonatal stridor, and choking. GPT-3.5 is primarily concerned with respiratory-related symptoms, such as throat swelling or tightness, cough or sore throat, difficulty breathing, hoarseness, and swallowing difficulties. GLM4, in addition to describing respiratory symptoms, mentions throat pain or discomfort, nasal congestion or runny nose, and emphasizes that laryngeal edema may lead to airway obstruction, causing difficulty breathing or even suffocation. In terms of treatment plans, the model in this paper offers a variety of comprehensive medications, including lysozyme tablets, acetylsalicylic acid tablets, levofloxacin hydrochloride capsules, and Isatis root granules, in addition to suggesting the avoidance of stimulating foods. In comparison, the other two models exhibit limited knowledge in specific drug recommendations.

Table 21 Comparison of responses from various models

Models	Question: Is laryngeal edema a complication of urticaria? If so, what are the symptoms of this complication?
GPT-3.5	Throat edema is usually not a direct complication of urticaria, but it may be associated with urticaria. Urticaria is a skin disease that typically manifests as red, swollen, itchy patches or papules on the skin, sometimes accompanied by a burning sensation. Throat edema may be part of an allergic reaction, and urticaria itself can also be a manifestation of allergies. If you have urticaria and experience swelling in the throat, you may experience the following symptoms: swelling or tightness of the throat, coughing or sore throat, difficulty breathing, hoarseness, and difficulty swallowing. In response to this situation, the first step is to ensure that the patient's respiratory tract is unobstructed. If serious symptoms such as difficulty breathing occur, immediate medical attention should be sought. Doctors may suggest using antihistamines to alleviate symptoms caused by urticaria and laryngeal edema. For severe allergic reactions, adrenaline injection may be necessary to quickly alleviate symptoms. However, treatment plans tailored to individual circumstances should be determined by doctors, as they can develop the most appropriate treatment plan based on specific circumstances
GPT-4	Yes, laryngeal edema can indeed be a complication of urticaria, especially when severe allergic reactions occur, such as anaphylactic shock or severe vascular edema. Throat edema is a serious condition that can be life-threatening. Its symptoms include: 1 Difficulty breathing: This is the most common and serious symptom, caused by swelling of the throat, narrowing of the airway, and obstruction of air entry and exit. 2. hoarseness: Due to swelling in the vocal cord area, the patient's voice may become hoarse or sound abnormal. 3. Difficulty swallowing: Swelling of the throat can also affect swallowing, and patients may feel throat blockage or pain during swallowing. 4. Tightness in the throat: Patients may feel tightness or compression in the throat. 5. Abnormal breathing sounds: Hearing wheezing (asthma like high pitched breathing sounds) or throat wheezing (low pitched breathing sounds), usually more pronounced during inhalation. 6. Swelling of the face, lips, or tongue: Sometimes it is not just swelling of the throat, but also swelling of the face, lips, or tongue, which can be part of a serious allergic reaction. Due to the rapid deterioration of laryngeal edema, which can lead to breathing difficulties, action must be taken as soon as possible. Here are some emergency response measures: 1 Seek medical attention immediately: Throat edema is an emergency, and patients should immediately go to the emergency department or call the emergency hotline. 2. Use of antihistamines: If antihistamines (such as diphenhydramine) are available, they can be taken on the way to the hospital, but this cannot replace professional medical treatment. 3. Adrenaline injection: For patients with a known history of severe allergies, it is important to carry and use an automatic adrenaline syringe (such as EpiPen) in a timely manner. In summary, laryngeal edema is a serious complication of urticaria that requires high attention and prompt treatment. Prevention and timely identification of symptoms are crucial for the safety of patients

Table 21 (continued)

Models	Question: Is laryngeal edema a complication of urticaria? If so, what are the symptoms of this complication?
GLM4	Throat edema can be a complication of urticaria, especially when urticaria is accompanied by vascular edema. Urticaria itself usually manifests as itching, red or skin colored bumps on the skin, while vascular edema is a deep swelling that may affect the mucosa, including the throat, leading to laryngeal edema. Throat edema is an emergency situation as it can lead to airway obstruction, breathing difficulties, and even suffocation. If laryngeal edema is caused by urticaria, it is usually part of an acute allergic reaction. The symptoms of laryngeal edema may include:—pain or discomfort in the throat—difficulty breathing or shortness of breath—hoarseness or loss of voice—difficulty swallowing—nasal congestion or runny nose (if the nasal cavity is also affected). Medications for treating laryngeal edema may include: 1. Epinephrine: This is a rapidly acting drug that can quickly reduce swelling and maintain airway patency. It is usually administered by injection, especially in emergency situations. 2. Corticosteroids, such as prednisone or dexamethasone, can reduce inflammation and swelling. 3. Antihistamines: can help alleviate symptoms of allergic reactions. If there is an infection present, antibiotics may be required. In the event of laryngeal edema, emergency medical assistance should be sought immediately. Doctors will determine the most appropriate treatment plan based on the severity and specific causes of the condition. In some cases, tracheostomy or other emergency surgery may be necessary to maintain airway patency
Ours	Throat edema is indeed a complication of urticaria. Urticaria is a common dermatological disease, characterized by symptoms such as severe itching, erythematous rash, and wheals. When urticaria is severe, it may cause complications such as laryngeal edema. The symptoms of laryngeal edema mainly include: 1 Cough hoarseness: Due to swelling of the mucous membrane in the throat, the voice becomes hoarse. 2. Difficulty breathing: Throat swelling may compress the airway, causing the patient to experience difficulty breathing. 3. Diffuse mucosal hypertrophy of the larynx: Swelling of the laryngeal mucosa, presenting with diffuse hypertrophy in appearance. 4. Neonatal laryngeal wheezing: The wheezing caused by laryngeal edema in newborns. 5. Asphyxia: Severe laryngeal edema may cause complete obstruction of the airway, leading to asphyxia. The drugs used to treat laryngeal edema include: 1 Lysozyme lozenges: have anti-inflammatory and anti-inflammatory effects, and can be used to treat laryngeal edema. 2. Acetylsparamycin tablets: antibiotics that can be used to treat bacterial infections of laryngeal edema. 3. Levofloxacin hydrochloride capsules: antibiotics with anti-inflammatory effects, can be used to treat laryngeal edema. 4. Banlangen granules: have the effects of clearing heat and detoxifying, reducing swelling and promoting pharynx, and can be used to treat laryngeal edema. The commonly used drugs for the treatment of urticaria include: 1 Loratadine capsules: antihistamines that can alleviate symptoms of urticaria. 2. Loratadine tablets: antihistamines that can also be used to alleviate symptoms of urticaria. 3. Fexofenadine hydrochloride tablets: antihistamines, suitable for the treatment of urticaria. 4. Fangfeng Tongsheng Pill: traditional Chinese patent medicines and simple preparations, with the functions of dispelling wind, relieving superficiality, clearing heat and detoxification, can be used for the treatment of urticaria. When treating urticaria and laryngeal edema, patients should avoid consuming irritating foods such as pig intestines, beer, pig tripe, and white beans. It is advisable to eat foods such as chicken liver, duck liver, chicken heart, and sesame. At the same time, patients should follow the doctor's advice, take medication on time, and undergo regular follow-up examinations. If you have emergency symptoms such as difficulty breathing, please seek medical attention promptly

Appendix E: All algorithms

Algorithm 1 Entity fast-linking Algorithm

Input: User input question S .

Output: Finally entity $entity_{final}$.

1. Define the set of disease $Entities = (e_1, e_2, \dots, e_n)$, user input sentence S , finally entities list $entity_{final}$, entity class $\mathcal{C} = \{C_1, C_2, \dots, C_n\}$, attributes of class $\mathcal{A}_i = \{a_{i1}, a_{i2}, \dots, a_{im_i}\}$, candidate entities $CEntities = (e'_1, e'_2, \dots, e'_m)$, Similarity List $simList$
 2. For each $Entities$ do
 3. If Multi-Pattern Matching(S, e_i)
 4. $entity_{final}.append(e_i)$
 5. End If
 6. End For
 7. If $len(entity_{final}) \neq 0$
 8. Return $entity_{final}$
 9. Else
 10. $f_i: E \rightarrow \mathcal{A}_i$
 11. $g_i(S) = \{a_{ij} \in \mathcal{A}_i \mid P(a_{ij}, S)\}$
 12. $\bigcap_{i=1}^n h_i(B_i(S)) \rightarrow CEntities$
 13. If $len(CEntities) > n$
 14. $word \leftarrow S$
 15. For each $CEntities$ do
 16. $simList \leftarrow word \otimes e'_i$
 17. End For
 18. Select the top L entities from $simList$
 19. Instruct $\rightarrow entity_{final}$
 20. Else
 21. Instruct $\rightarrow entity_{final}$
 22. End If
 23. End If
 24. Return $entity_{final}$
-

Algorithm 2 n -Hop Subgraph E Construction Algorithm

Input: *entity*, *current_depth*, max depth n .

Output: n -hop full path knowledge triplet.

1. Initialize parameters *visited*, *path* and *pathDict*.
 2. If *entity* is in *visited* or *current_depth* $\geq n$
 3. Return *pathDict*
 4. If *path* is not empty
 5. *pathDict*[startNode] $\leftarrow$ *path*
 6. End if
 7. End if
 8. Add *entity* to *visited*
 9. Retrieve relationships from the Neo4j $\rightarrow$ *result*
 10. If $\text{len}(\text{result}) == 0$
 11. If *path* is not empty
 12. *pathDict*[startNode] $\leftarrow$ *path*.
 13. Return *pathDict*
 14. End if
 15. End if
 16. For each *result*
 17. *path.append(triple)*
 18. *pathDict* $\leftarrow$ Recursively call function.
 19. Clear *path*
 20. End for
 21. Return *pathDict*
-

Algorithm 3 Knowledge Fusion Algorithm

Input: i -th knowledge triple $level_i$
Output: i -th knowledge path $levelKP_i$.

1. $level_i \leftarrow path[entity]$
2. Initialize parameters $levelKP_i$, T_{list} , r_{prev} , h_{prev} , $count$
3. $\mathbb{P}(e) \leftarrow (h, r, t)$
4. For each $\mathbb{P}(e)$
5. If $r \neq r_{prev}$ and $count \neq 0$
6. $h_{prev}, r_{prev}, T_{list} \rightarrow levelKP_i$
7. $T_{list} \rightarrow \emptyset$
8. $T_{list} \leftarrow t$
9. If $count = |\mathbb{P}(e)| - 1$
10. $l_1 \leftarrow h_{prev}, r_{prev}, T_{list}$
11. $l_i \rightarrow levelKP_i$
12. End if
13. Else
14. $T_{list} \leftarrow t$
15. $h_{prev} \leftarrow h$
16. $r_{prev} \leftarrow r$
17. $count += 1$
18. End for

Algorithm 4 Subgraph E pruning Algorithm

Input: Subgraph E

Output: Intent graph after pruning E' .

1. $simpleLevelKP_i \leftarrow LevelKP_i$
 2. $judgeResult \leftarrow \mathcal{P}_9$ or $\mathcal{P}_{10}$
 3. If $judgeResult == 'YES'$
 4. $E' \leftarrow LevelKP_i$
 5. Return E'
 6. End if
 7. Else
 8. $textPath \leftarrow LevelKP_i$
 9. $getNextPath(textPath) \rightarrow level_j$
 10. If $depth < n$
 11. Call Algorithm 4 with updated parameters
 12. Else
 13. $level_j \rightarrow LevelKP_j$
 14. $E' \leftarrow LevelKP_j$
 15. Return E'
 16. End Else
-

Acknowledgements This work was supported by Mass Spectrometry Key Technology R&D and Clinical Application of Anhui Province Jointly Constructed Discipline Key Experiments (GRANT: 2023ZPLH07); Humanity and Social Science Research Project of Anhui Provincial Education Department (GRANT: 2022AH050224); Housing urban and rural construction science and technology plan project of Anhui Province in 2022 (GRANT: 2022-YF082); Director's Fund of the Anhui Province Key Laboratory of Intelligent Building and Building Energy Saving, Anhui Jianzhu University (Grant No. IBES2024ZR01).

Author contributions Wang was responsible for writing the paper and conducting the experiments. Shi and Jose provided theoretical guidance for the paper. Cui contributed ideas for the experiments. Shen and Li were responsible for reviewing the paper.

Data availability The data used in this work was taken from (<https://github.com/liuhuanyong/QASystemOnMedicalKG>) and is described in Appendix B.

Code availability The code developed during this project is available at (<https://github.com/wfl0325/LLM-KGMQA>).

Declarations

Conflict of interest The authors declare no competing interests.

References

1. Adlakha V, Behnamghader P, Lu XH et al (2023) Evaluating correctness and faithfulness of instruction-following models for question-answering. [arXiv:2307.16877](https://arxiv.org/abs/2307.16877)

2. Andrus BR, Nasiri Y, Cui S et al (2022) Enhanced story comprehension for large language models through dynamic document-based knowledge graphs. In: Proceedings of the AAAI conference on artificial intelligence, pp 10436–10444
3. Bordes A, Usunier N, Garcia-duran A et al (2013) Translating embeddings for modeling multi-relational data. *Advances in neural information processing systems*, vol 26
4. Cai S, Wu X, Maimaiti M et al (2023) An adaptive mixup hard negative sampling for zero-shot entity linking. *Mathematics* 11:4366
5. Choudhary N, Reddy CK (2023) Complex logical reasoning over knowledge graphs using large language models. [arXiv:2305.01157](https://arxiv.org/abs/2305.01157)
6. Chowdhery A, Narang S, Devlin J et al (2023) Palm: scaling language modeling with pathways. *J Mach Learn Res* 24(240):1–113
7. Darius F, Faridz I, Zhuyan C et al (2023) Few-shot entity linking of food names. *Inf Process Manag* 60:103463
8. Dettmers T, Pagnoni A, Holtzman A et al (2024) Qlora: efficient finetuning of quantized llms. *Advances in neural information processing systems*, vol 36
9. Dong L, Wei F, Zhou M et al (2015) Question-answering over freebase with multi-column convolutional neural networks. In: Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th international joint conference on natural language processing (Volume 1: Long papers), pp 260–269
10. Gao Y, Li R, Caskey J et al (2023) Leveraging a medical knowledge graph into large language models for diagnosis prediction. [arXiv:2308.14321](https://arxiv.org/abs/2308.14321)
11. Gardner M, Talukdar P, Kisiel B et al (2013) Improving learning and inference in a large knowledge-base using latent syntactic cues. In: Proceedings of the 2013 conference on empirical methods in natural language processing, pp 833–838
12. Harsha Vardhan LV, Jia G, Kok S (2020). Probabilistic logic graph attention networks for reasoning. In: Companion proceedings of the web conference, vol 2020, pp 669–673
13. He J, Zhou C, Ma X et al (2021) Towards a unified view of parameter-efficient transfer learning. [arXiv:2110.04366](https://arxiv.org/abs/2110.04366)
14. Houslyby N, Giurgiu A, Jastrzebski S et al (2019) Parameter-efficient transfer learning for NLP. In: International conference on machine learning. PMLR, pp 2790–2799
15. Hu E J, Shen Y, Wallis P et al (2021) Lora: low-rank adaptation of large language models. [arXiv:2106.09685](https://arxiv.org/abs/2106.09685)
16. Jiang P, Agarwal S, Jin B et al (2023) Text-augmented open knowledge graph completion via pre-trained language models. [arXiv:2305.15597](https://arxiv.org/abs/2305.15597)
17. Jiang X, Zhang R, Xu Y et al (2023) HyKGE: a hypothesis knowledge graph enhanced framework for accurate and reliable medical LLMs responses. [arXiv:2312.15883](https://arxiv.org/abs/2312.15883)
18. Jingchao W, Weimin L, Fangfang L et al (2023) Hic-KGQA: improving multi-hop question-answering over knowledge graph via hypergraph and inference chain. *Knowl Based Syst* 277:110810
19. Kirkpatrick J, Pascanu R, Rabinowitz N et al (2017) Overcoming catastrophic forgetting in neural networks. *Proc Natl Acad Sci* 114(13):3521–3526
20. Kuokun T, He C, Charles C et al (2021) A secure live signature verification with Aho-Corasick histogram algorithm for mobile smart pad. *Electronics* 10:1337–1337
21. Lester B, Al-rfou R, Conatant N (2021) The power of scale for parameter-efficient prompt tuning. [arXiv:2104.08691](https://arxiv.org/abs/2104.08691)
22. Li XL, Liang P (2021) Prefix-tuning: optimizing continuous prompts for generation. [arXiv:2101.00190](https://arxiv.org/abs/2101.00190)
23. Liu T, Zhang Y, Brockett C et al (2021) A token-level reference-free hallucination detection benchmark for free-form text generation. [arXiv:2104.08704](https://arxiv.org/abs/2104.08704)
24. Liu X, Ji K, Fu Y et al (2021) P-Tuning v2: prompt tuning can be comparable to fine-tuning universally across scales and tasks. [arXiv:2110.07602](https://arxiv.org/abs/2110.07602)
25. Liu X, Zheng Y, Du Z et al (2023). GPT understands, too. *AI Open*
26. Lv X, Lin Y, Cao Y et al (2022) Do pre-trained models benefit knowledge graph completion? A reliable evaluation and a reasonable approach. In: Findings of the Association for Computational Linguistics: ACL 2022. pp 3570–3581
27. Ming HJ, Jie WX, Yuan L et al (2024) The biomedical knowledge graph of symptom phenotype in coronary artery plaque: machine learning-based analysis of real-world clinical data. *BioData Min* 17:13
28. Mingyang L, Yuqing X, Fang K et al (2021) Towards better entity linking. *Front Comput Sci* 16:162308
29. Peng B, He W, Chen B et al (2024) Multi-perspective thought navigation for source-free entity linking. *Pattern Recogn Lett* 178:84–90
30. Pengxuan Z, Yuan Y, Lei C (2023) ELECTRA-based graph network model for multi-hop question-answering. *J Intell Inf Syst* 61:819–834

31. Pfeiffer J, Vulić I, Gurevych I et al (2020) Mad-x: an adapter-based framework for multi-task cross-lingual transfer. [arXiv:2005.00052](#)
32. Ren H, Leskovec J JAINIPS (2020) Beta embeddings for multi-hop logical reasoning in knowledge graphs. *Adv Neural Inf Process Syst* 33:19716–19726
33. Ruder S, Peters ME, Swayamdipta S et al (2019) Transfer learning in natural language processing. In: *Proceedings of the 2019 conference of the North American chapter of the Association for Computational Linguistics: tutorials*, pp 15–18
34. Saxena A, Tripathi A, Talukdar P (2020) Improving multi-hop question-answering over knowledge graphs using knowledge base embeddings. In: *Proceedings of the 58th annual meeting of the Association for Computational Linguistics*, pp 4498–4507
35. Schlichtkrull M, Kipf TN, Bloem P et al (2018) Modeling relational data with graph convolutional networks. In: *The semantic web: 15th international conference, ESWC 2018, Heraklion, Crete, Greece, June 3–7, 2018, proceedings 15*. Springer, pp 593–607
36. Shi Y, Yang R, Yin C et al (2023) Entity linking method for chinese short texts with multiple embedded representations. *Electronics* 12:2692
37. Shu D, Chen T, Jin M et al (2024) Knowledge graph large language model (KG-LLM) for link prediction. [arXiv:2403.07311](#)
38. Singh A, Krishnamoorthy S, Ortega JE (2024) NeighBERT: medical entity linking using relation-induced dense retrieval. *J Healthc Inform Res* 8:353–369
39. Tang Z, Zhou K, Wang P et al (2023) Detoxify language model step-by-step. [arXiv:2308.08295](#)
40. Wang Y, Wang L, Li Y, He D, Liu T (2013) A theoretical analysis of NDCG type ranking measures. In: *Proceedings of the 26th annual conference on learning theory*, vol 30, pp 25–54
41. Wang M, Zhang N, Xu Z et al (2024) Detoxifying large language models via knowledge editing. [arXiv:2403.14472](#)
42. Wang X, Gao T, Zhu Z et al (2021) KEPLER: A unified model for knowledge embedding and pre-trained language representation. *Trans Assoc Comput Linguist* 2021(9):176–194
43. Wang X, Zhou W, Zu C et al (2023). InstructUIE: multi-task instruction tuning for unified information extraction. [arXiv:2304.08085](#)
44. Wei J, Tay Y, Bommasani R et al (2022) Emergent abilities of large language models. [arXiv:2206.07682](#)
45. Wei J, Wang X, Schuurmans D et al (2022) Chain-of-thought prompting elicits reasoning in large language models. In: *36th conference on neural information processing systems, NeurIPS 2022, November 28, 2022–December 9, 2022*. New Orleans, LA, United States: neural information processing systems foundation.
46. Wei X, Ccui X, Cheng N et al (2023) Zero-shot information extraction via chatting with chatgpt. [arXiv:2302.10205](#)
47. Wen Y, Wang Z, Sun J (2023) Mindmap: knowledge graph prompting sparks graph of thoughts in large language models. [arXiv:2308.09729](#)
48. Xiaolong L, Shuifa S, Tinglong T et al (2023) Construction of a knowledge graph for breast cancer diagnosis based on Chinese electronic medical records: development and usability study. *BMC Med Inform Decis Mak* 23:210–210
49. Xiong W, Hoang T, Wang WY (2017). DeepPath: a reinforcement learning method for knowledge graph reasoning. In: *Proceedings of the 2017 conference on empirical methods in natural language processing*, pp 564–573
50. Xu D, Zhang Z, Lin Z et al (2024) Multi-perspective improvement of knowledge graph completion with large language models. [arXiv:2403.01972](#)
51. Yang C, Weibing W, Yuming Z et al (2023) Generalization performance optimization of KBQA system for Chinese open domain. *Multimed Tools Appl* 83:12445–12466
52. Yasunaga M, Bosselut A, Ren H et al (2022) Deep bidirectional language-knowledge graph pretraining. *Adv Neural Inf Process Syst* 35:37309–37323
53. Zhang Q, Chen M, Bukharin A et al (2023) Adaptive budget allocation for parameter-efficient fine-tuning. In: *International conference on learning representations*. Openreview
54. Zhang W, Kong L, Lee S et al (2024) Detecting mental and physical disorders using multi-task learning equipped with knowledge graph attention network. *Artif Intell Med* 149:102812
55. Zhe C, Yuehan W, Bin Z et al (2020) Knowledge Graph completion: a review. *IEEE ACCESS*. 8:192435–192456
56. Zhibin C, Yuting W, Yansong F et al (2022) Integrating manifold knowledge for global entity linking with heterogeneous graphs. *Data Intell* 4:20–40
57. Junchen G, Yushang M, Yan X et al (2024) An entity linking method based on prompt scoring. *Comput Eng* 1–11

58. Jian P, Zhiwei W, Yanjun L (2024) CGR-BERT-ZESHEL: zero-shot entity linking model with Chinese features. *Comput Sci* 1–11
59. Heyi Z, Xin W, Lifan H et al (2023) Research on question-answering system on joint of knowledge graph and large language model. *J Front Comput Sci Technol* 17:2377–2388

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



FeiLong Wang was born in 2000 and is a M.S. candidate. His research interests include large language models, natural language processing and knowledge graph.



Donghui Shi is a professor in the Department of Computer Engineering, School of Electronics and Information Engineering, Anhui Jianzhu University, China. He received his Ph.D. in computer science from the University of Science and Technology of China. He was a visiting scholar in the Department of Computer and Information Systems, School of Business, University of Louisville in the USA from 2012.2 to 2013.3. He worked as Prometeo professor at Universidad Técnica Particular de Loja in Ecuador from 2014.10 to 2015.11. His research interests focus on machine learning and data mining, especially in ontology, data stream, and text mining. His recent work applies data-mining algorithms to solve practical problems in aviation safety, construction safety, real estate and biomedicine. He published papers in *JMIS*, *ESWA*, *EAAI* and *WIREs DMKD*.



Jose Aguilar received the Systems Engineer degree from the Universidad de Los Andes, Mérida, Venezuela, in 1987; the M.Sc. degree in computer science from Université Paul Sabatier-France, in 1991; and the Ph.D. degree in computer science from Université René Descartes-France, in 1995. He has carried out several Postdoctorals postdoctorals at the Department of Computer Science, University of Houston, USA, 1999–2000; the Laboratoire d'analyse et d'architecture des systèmes (LAAS), CNRS, Toulouse, France, 2011–2012; and the University of Alcalá, Spain, 2020–2022, with a Marie Skłodowska-Curie Fellowship. He is currently a Senior Research at the IMDEA Networks Institute, Spain. He is also a member of the Mérida Science Academy, Emeritus Professor at the Department of Computer Science, Universidad de Los Andes, and was member of the IEEE CIS Technical Committee on Neural Networks and President of Latin American Center for Computer Science Studies. He has published more than 700 articles and ten books, in the field of parallel and distributed computing, computer intelligence, and science and technology management. His research

interests include artificial intelligence, semantic mining, big data, emerging computing, and intelligent environments.



Aguilar Cui was born in 2000 and is a M.S. candidate. Her research interests include large language models and machine learning.



Jinsong Jiang was born in 1999 and is a M.S. candidate. His research interests include multi-modal knowledge graph and natural language processing.



Longjian Shen was born in 2000 and is a M.S. candidate. His research interests include large language models and machine learning.



Mengya Li was born in 2001 and is a M.S. candidate. Her research interests include large language models and artificial intelligence.

Authors and Affiliations

FeiLong Wang^{1,2} · Donghui Shi^{1,2} · Jose Aguilar^{3,4,5} · Xinyi Cui¹ · Jinsong Jiang^{1,2} · Longjian Shen¹ · Mengya Li¹

✉ Donghui Shi
shidonghui@ahjzu.edu.cn

FeiLong Wang
wfl0325@stu.ahjzu.edu.cn

Jose Aguilar
aguilar@ula.ve

Xinyi Cui
Cxy2000321972@163.com

Jinsong Jiang
jjs@stu.ahjzu.edu.cn

Longjian Shen
202333105124@stu.ahjzu.edu.cn

Mengya Li
mengya_li03@163.com

- ¹ Department of Computer Engineering, School of Electronics and Information Engineering, Anhui Jianzhu University, Hefei 230601, China
- ² Mass Spectrometry Key Technology R&D and Clinical Application of Anhui Province Jointly Constructed Discipline Key Experiments, Hefei 230601, China
- ³ Grupo de Investigación en I+D+i en TIC, Universidad EAFIT, Medellín, Colombia
- ⁴ Centro de Estudios en Microelectrónica y Sistemas Distribuidos, Universidad de Los Andes, Mérida, Venezuela
- ⁵ IMDEA Networks Institute, Madrid, Spain